SOFTWARE ENGINEERING

Complete Question Bank with Full Solutions

2021 · 2023 · 2024 · 2025 Spring · 2025 First Term/Pre-University

Provided By: Gajendra Awasthi *Email: 24530017@nast.edu.np*

Questions from 2021 Examinations

Q1. Define Software Engineering. List out some roles of Software Engineering, discuss attributes/characteristics of a good software, and explain its purpose and advantages in real life. *[8 Marks]*

In Simple English

Software Engineering is the disciplined, organized way of building software — instead of just writing code randomly, it uses planning, design, and testing steps to make sure the final product actually works well and keeps working.

Definition

Software Engineering is a systematic, disciplined, and quantifiable approach to the design, development, operation, and maintenance of software, applying engineering principles to produce reliable and efficient software within budget and schedule constraints.

Roles in Software Engineering

- **Requirements Analyst:** Gathers and documents what the client/user actually needs.
- **Software Designer/Architect:** Designs the system's structure, modules, and interfaces.
- **Programmer/Developer:** Writes and implements the actual code.
- **Tester/QA Engineer:** Verifies the software meets requirements and is free of defects.
- **Project Manager:** Plans, schedules, and coordinates the overall project.
- **Maintenance Engineer:** Fixes bugs and adds enhancements after release.

Attributes/Characteristics of Good Software

- **Functionality:** Performs the tasks it was designed for, correctly.
- **Reliability:** Operates without failure under stated conditions for a specified period.
- **Usability:** Easy to learn and operate for its intended users.
- **Efficiency:** Uses system resources (memory, CPU, time) economically.
- **Maintainability:** Can be modified easily to fix defects or add features.
- **Portability:** Can run on different platforms/environments with minimal change.
- **Security:** Protects data and resources from unauthorized access.
- **Correctness:** Meets its specified requirements precisely.

Purpose and Advantages in Real Life

- Ensures software is delivered on time, within budget, and to specification — reducing costly project failures
- Produces higher-quality, more reliable software through structured processes (requirements, design, testing)
- Makes large, complex software systems manageable by breaking them into smaller, well-defined modules
- Improves long-term maintainability, reducing the cost and risk of future changes
- Provides reusable processes and tools (e.g., design patterns, CASE tools) that increase productivity across projects

Q2. What do you mean by software specification? Explain different types of software specifications and their uses/role for quality of software development. *[8 Marks]*

In Simple English

A software specification is a clear, written description of exactly what the software must do and how well it must do it — like a detailed blueprint that everyone (client, designers, testers) can agree on and refer back to.

Technical Explanation

A software specification (Software Requirements Specification, SRS) is a formal document that precisely describes the functions, features, constraints, and behavior of a software system, serving as the primary agreement between stakeholders and the development team about what will be built.

Types of Software Specifications

- **Functional Specification:** Describes what the system should do — specific functions, features, and behaviors (e.g., 'the system shall allow users to reset their password').
- **Non-Functional Specification:** Describes quality attributes/constraints such as performance, security, usability, and reliability (e.g., 'the system shall respond within 2 seconds').
- **Formal Specification:** Uses mathematical notation to precisely and unambiguously define system behavior, useful for safety-critical systems.
- **Informal (Natural Language) Specification:** Written in plain language, easy for stakeholders to understand but more prone to ambiguity.
- **Interface Specification:** Defines how the software interacts with other systems, hardware, or components (APIs, protocols).

Role/Uses for Quality of Software Development

- Provides a shared, unambiguous understanding between clients and developers, reducing miscommunication
- Serves as the baseline against which testing and validation are performed, ensuring the delivered product matches what was promised
- Helps in accurate project estimation (cost, time, resources) since scope is clearly defined upfront
- Reduces costly rework caused by misunderstood or changing requirements late in development
- Acts as a legal/contractual reference in case of disputes about project scope

Q3. Define Test Case. Differentiate between Integration Testing and System Testing, highlighting their importance, advantages, and disadvantages. *[8 Marks]*

In Simple English

A test case is a specific, written check — 'if I do X, I expect Y to happen' — used to verify one particular behavior of the software works correctly.

Definition

A test case is a documented set of conditions, input values, execution steps, and expected results, designed to verify that a specific feature or requirement of the software functions correctly.

Integration Testing vs System Testing

Aspect	Integration Testing	System Testing
Definition	Tests the interfaces and interaction between two or more integrated modules/components	Tests the complete, fully-integrated system as a whole against its requirements
Scope	Focused on module-to-module interfaces	Entire end-to-end system, including hardware/environment
When Performed	After unit testing, before system testing	After integration testing, before acceptance testing
Importance	Catches interface/data-flow errors between modules early	Validates that the overall system meets business and functional requirements
Advantages	Detects interface defects early; cheaper to fix at this stage	Confirms the system works correctly in a production-like environment
Disadvantages	May miss defects that only surface with the full system	Expensive/time-consuming; defects found late are costlier to fix

Q4. Define the terms software quality and Software Quality Assurance (SQA). Discuss the activities performed by the SQA team, describe its importance/significance, explain any two quality standards, and elaborate on the quality characteristics of the ISO model. *[8 Marks]*

In Simple English

Software quality means the software actually does what it's supposed to, reliably and without annoying bugs. SQA is the ongoing set of activities and checks that make sure the whole development process stays on track to produce that quality.

Definitions

Software Quality: The degree to which a software product meets its specified requirements, user needs, and expected quality attributes (reliability, usability, efficiency, etc.).

Software Quality Assurance (SQA): A planned and systematic set of activities that ensure software processes and products conform to established standards and procedures, aiming to build quality into the software throughout its development, rather than just testing for it at the end.

Activities Performed by the SQA Team

- Preparing and applying SQA plans for each project
- Reviewing and auditing software work products (design docs, code, test plans) for conformance to standards
- Ensuring deviations in software processes/products are documented and handled per a pre-defined procedure
- Recording and reporting any non-compliance to management
- Verifying that reviews (technical/management) are conducted properly
- Coordinating change control and configuration management

Importance/Significance of SQA

- Detects defects and process deviations early, reducing the cost of fixing them later
- Builds confidence in the reliability and correctness of the delivered product
- Ensures consistency and adherence to organizational/industry standards across projects
- Improves customer satisfaction by reducing post-release defects

Two Quality Standards

- **ISO 9001:** A generic quality-management standard applicable to any organization, defining requirements for a quality management system, including software development processes.
- **CMMI (Capability Maturity Model Integration):** A process-improvement framework that assesses and ranks an organization's software development maturity across 5 levels, from ad-hoc (Initial) to fully optimized (Optimizing).

Quality Characteristics of the ISO 9126 Model

Characteristic	Description
Functionality	Presence of functions that satisfy stated/implied needs
Reliability	Ability to maintain performance under stated conditions over time
Usability	Effort required to learn and use the software
Efficiency	Performance relative to resources used
Maintainability	Effort required to make modifications
Portability	Ability to transfer the software to different environments

---

Q5. What do you mean by Defect Amplification and Removal? Explain the importance of this model in quality assurance, and explain the cost of software defect using this model. *[8 Marks]*

In Simple English

If a mistake is made early (like in requirements) and isn't caught, it doesn't just stay one mistake — it multiplies as it flows into design, then code, then testing, causing more and more problems (and costing more to fix) the longer it goes unnoticed.

Technical Explanation

The Defect Amplification and Removal model illustrates how errors introduced at one phase of development (e.g., requirements) can be passed on to subsequent phases (design, coding, testing) where they may be amplified — a single requirements error might result in multiple design errors, which in turn produce even more coding errors — unless they are detected and removed at the phase they occur.

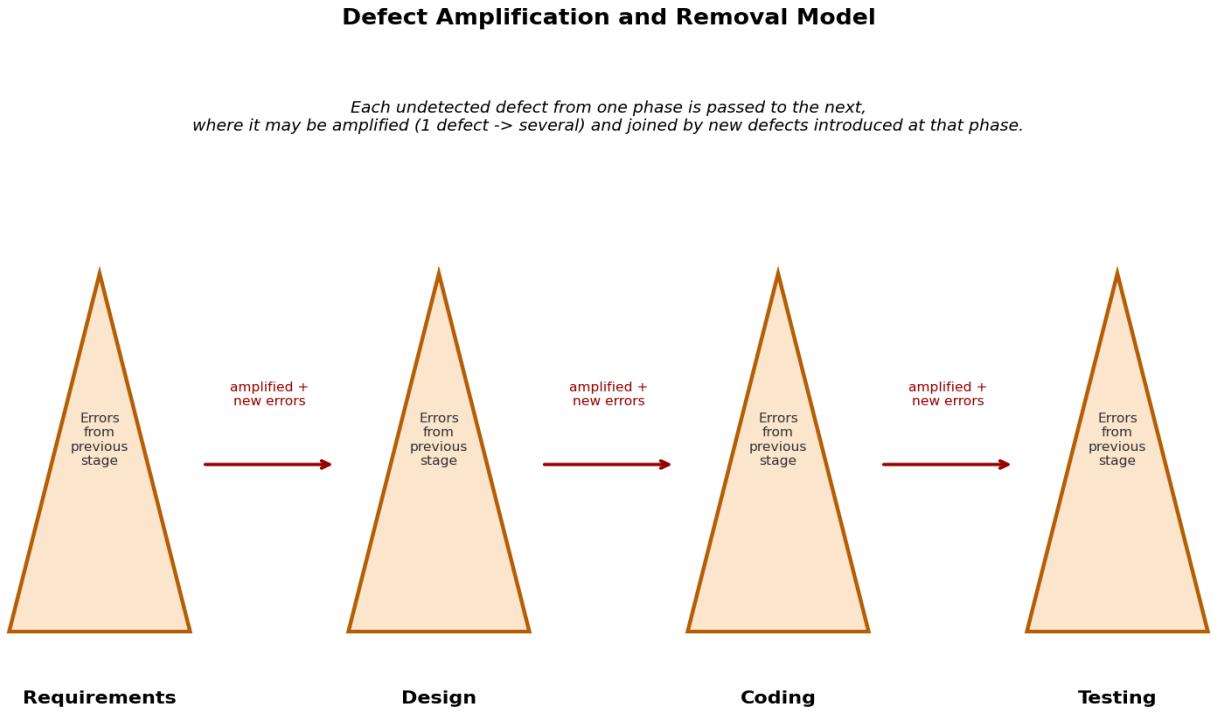

Figure: Defect Amplification and Removal Model

Importance in Quality Assurance

- Emphasizes the need for reviews and inspections at every phase, not just final testing
- Highlights that early defect detection (e.g., during requirements review) is far more valuable than catching the same defect after it has propagated
- Guides SQA teams in allocating review/testing effort to the earliest phases of the lifecycle

Cost of Software Defect Using This Model

The model shows that the cost of fixing a defect increases (often exponentially) the later it is discovered in the lifecycle. A defect that costs 1 unit to fix if found during requirements review might cost 10 units during design, 100 units during coding/testing, and far more (potentially 1000+ units) if discovered only after the software is deployed to production. This strongly motivates 'shifting left' — finding and fixing defects as early as possible.

Q6. Considering some examples, express your views on "Software reliability is an important quality attribute." *[8 Marks]*

In Simple English

If software keeps crashing or giving wrong answers, nobody will trust or use it — no matter how many features it has. Reliability is often the single most important thing users actually care about.

Discussion

Software reliability is defined as the probability that software will operate without failure for a specified period under specified conditions. It is arguably the most critical quality attribute because:

- **Safety-Critical Systems:** In systems like aircraft flight control, medical devices (pacemakers), or nuclear plant control software, a single failure can cause loss of life — reliability is non-negotiable.
- **Financial Systems:** Banking and stock-trading software failures can cause massive financial losses within seconds (e.g., an unreliable trading algorithm executing incorrect trades).
- **User Trust:** Even in everyday consumer apps, frequent crashes or data loss quickly destroy user trust and adoption, regardless of how feature-rich the software is.
- **Cascading Failures:** In interconnected systems (e.g., cloud services), one unreliable component can trigger failures across many dependent systems.

While attributes like usability or aesthetics affect user satisfaction, reliability directly affects whether the software can be trusted and used at all — making it foundational among all quality attributes.

Q7. Why is project estimation necessary? Explain the Basic COCOMO model for project estimation with examples in detail. *[8 Marks]*

In Simple English

Before starting a software project, you need a good guess of how long it will take and how many people it will need — otherwise you can't set a realistic budget, deadline, or team size.

Why Project Estimation Is Necessary

- Enables realistic budgeting and resource allocation before development begins
- Helps set achievable deadlines and manage stakeholder expectations
- Supports decisions on team size and required skill sets
- Provides a baseline to track actual progress against planned progress
- Reduces risk of project failure due to underestimation of effort/cost

Basic COCOMO (Constructive Cost Model)

Basic COCOMO estimates software development effort (in person-months) as a function of the estimated project size in KLOC (thousands of lines of code), using the formula:

```
Effort (E) = a × (KLOC)^b   person-months
Development Time (D) = c × (E)^d   months
```

The constants a, b, c, d depend on the project mode/category:

Project Mode	a	b	c	d	Description
Organic	2.4	1.05	2.5	0.38	Small, simple, experienced team, familiar environment
Semi-Detached	3.0	1.12	2.5	0.35	Medium size/complexity, mixed team experience
Embedded	3.6	1.20	2.5	0.32	Complex, tight constraints (hardware, real-time), novel

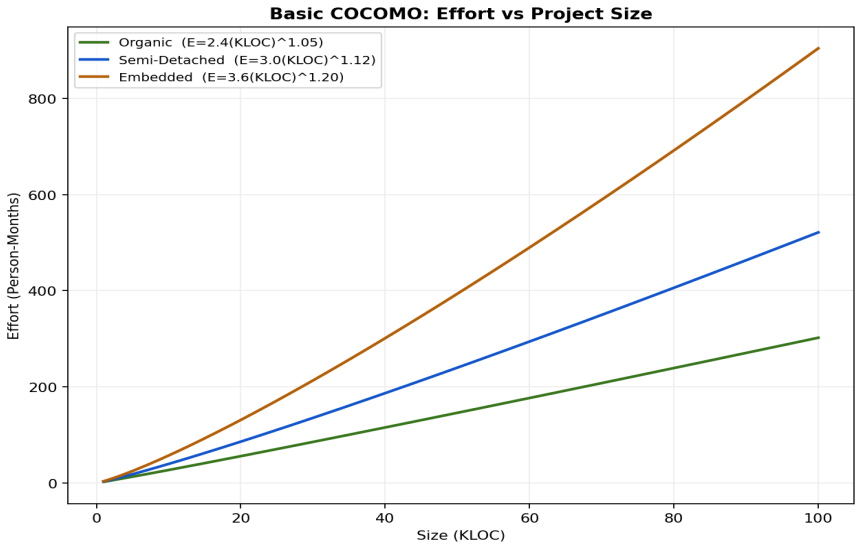

Figure: Basic COCOMO Effort vs Project Size

Example

Suppose a project is estimated at 30 KLOC and falls into the Organic mode:

```
E = 2.4 × (30)^1.05 = 2.4 × 34.85 ≈ 83.6 person-months
D = 2.5 × (83.6)^0.38 = 2.5 × 5.6 ≈ 14 months
Average staffing = E / D = 83.6 / 14 ≈ 6 people
```

This tells the project manager that the project will need approximately 84 person-months of effort, completed over about 14 months, requiring an average team of roughly 6 people.

Q8. Differentiate between COM and DCOM. *[8 Marks]*

In Simple English

COM lets different pieces of software on the same computer talk to each other and share functionality. DCOM does the same thing, but lets them talk across different computers over a network.

Aspect	COM (Component Object Model)	DCOM (Distributed COM)
Scope	Enables communication between software components on the same machine	Extends COM to enable communication between components across a network on different machines
Location Transparency	Not required (components are local)	Provides location transparency — a client need not know whether the component is local or remote
Communication	In-process or local inter-process calls	Uses remote procedure calls (RPC) over a network
Performance	Faster (no network overhead)	Slower due to network latency and marshaling overhead
Use Case	Single-machine applications, e.g., plugin architectures	Distributed enterprise applications spanning multiple servers

Q9. List out the jobs/responsibilities of a software project manager, and explain the SPMP document. *[8 Marks]*

In Simple English

A project manager is responsible for making sure the whole project runs smoothly — planning it, tracking progress, managing the team, and handling problems as they arise. The SPMP is the master planning document that records all of this.

Responsibilities of a Software Project Manager

- Project planning — defining scope, schedule, budget, and resource requirements
- Risk management — identifying, analyzing, and mitigating project risks
- Team management — assigning tasks, monitoring performance, resolving conflicts
- Progress tracking and reporting to stakeholders/management
- Quality assurance oversight, ensuring processes/standards are followed
- Communication with clients and stakeholders regarding requirements and status
- Change management — handling scope changes and their impact on schedule/budget

Software Project Management Plan (SPMP) Document

The SPMP is a formal document that describes how a software project will be planned, organized, staffed, monitored, and controlled. It typically includes:

- Project overview and objectives
- Project organization and team structure/roles
- Managerial process plans — risk management, schedule/budget control, staffing plans
- Technical process plans — process model, methods, tools, and techniques to be used
- Work packages, schedule, and resource/budget allocation

Q10. Describe functional testing, structural testing, domain testing, and non-functional testing in detail. *[8 Marks]*

- **Functional Testing:** Also called black-box testing — verifies that the software performs its specified functions correctly, testing inputs against expected outputs without regard to internal code structure.
- **Structural Testing:** Also called white-box/glass-box testing — examines the internal code structure, logic paths, and branches to ensure all code paths are exercised and behave correctly.
- **Domain Testing:** Focuses on testing input values from the valid and invalid ranges (domains) of each input variable, including boundary values, to ensure the software handles the full spectrum of possible inputs correctly.
- **Non-Functional Testing:** Verifies quality attributes rather than specific functions — such as performance, load capacity, security, usability, and reliability — ensuring the software meets its quality-of-service requirements, not just its functional requirements.

Q11. From the given values, compute Function Point (F.P) when all complexity adjustment factors are average (i.e., 3) and weighting factors are average. Given: User Input=50, User Output=40, User Inquiries=35, User Files=6, External Interface=4. ***[8 Marks]***

Step 1: Assign Average Weights (Standard IFPUG Weights)

Component	Count	Average Weight	Weighted Count
External Inputs (User Input)	50	4	200
External Outputs (User Output)	40	5	200
External Inquiries (User Inquiries)	35	4	140
Internal Logical Files (User Files)	6	10	60
External Interface Files	4	7	28

Step 2: Compute Count Total (CT)

```
CT = 200 + 200 + 140 + 60 + 28 = 628
```

Step 3: Compute the Complexity Adjustment Factor

With 14 General System Characteristics (GSCs), each rated at the average value of 3:

```
ΣFi = 14 × 3 = 42
CAF = 0.65 + (0.01 × ΣFi) = 0.65 + 0.42 = 1.07
```

Step 4: Compute Function Points (FP)

```
FP = CT × CAF = 628 × 1.07 = 671.96
FP ≈ 672 Function Points
```

Questions from 2023 Examinations

Q1. Describe the challenges of software engineering in software development. *[8 Marks]*

In Simple English

Building software is genuinely hard — requirements change, deadlines are tight, and mistakes are easy to make, so software engineers constantly face real obstacles while trying to deliver good software.

Technical Explanation — Major Challenges

- **Changing Requirements:** Client needs often evolve during development, making it difficult to build against a moving target.
- **Time and Budget Constraints:** Projects are frequently expected to be delivered faster and cheaper than realistically possible.
- **Managing Complexity:** Modern systems are large and intricate, with many interacting components, making them hard to design, test, and maintain.
- **Legacy System Integration:** New software often must work alongside old, poorly-documented systems.
- **Quality Assurance:** Ensuring reliability, security, and performance under real-world conditions is difficult and resource-intensive.
- **Skill Shortages & Team Coordination:** Finding skilled developers and coordinating distributed/remote teams effectively.
- **Rapid Technology Change:** Keeping pace with constantly evolving tools, frameworks, and platforms.
- **Security Threats:** Increasing cyber-security risks demand continuous vigilance throughout development.

Q2. Define requirement in software projects. What do you mean by functional and non-functional requirements? Define Load Testing and Stress Testing. *[8 Marks]*

In Simple English

A requirement is simply a statement of something the software must do or be. Functional requirements describe what the system does; non-functional requirements describe how well it does it.

Definition of Requirement

A requirement is a documented condition or capability that a software system must satisfy or possess, either to solve a real-world problem or to meet a contract, standard, or specification imposed on the project.

Functional vs Non-Functional Requirements

Aspect	Functional Requirements	Non-Functional Requirements
Definition	Describe specific functions/features the system must perform	Describe quality attributes/constraints on how the system performs
Example	'The system shall allow users to log in with a username and password'	'The login process shall complete within 2 seconds'
Focus	What the system does	How well the system performs
Verification	Verified through functional/black-box testing	Verified through performance, load, security testing

Load Testing

Load testing evaluates how a system behaves under an expected or peak workload (e.g., a specific number of concurrent users), verifying that the system can handle anticipated real-world usage without performance degradation.

Stress Testing

Stress testing evaluates system behavior beyond normal or peak load — pushing the system past its limits to determine its breaking point and observe how it fails and recovers, ensuring it degrades gracefully rather than crashing catastrophically.

Q3. Compare Alpha Testing and Beta Testing, and outline their respective advantages and disadvantages. *[8 Marks]*

In Simple English

Alpha testing happens inside the company, before the product goes anywhere near real customers. Beta testing happens after that, when a limited group of actual customers try it out in their own real-world environment.

Aspect	Alpha Testing	Beta Testing
Performed By	Internal developers/testers at the developer's site	Real end-users/customers at their own site
Environment	Controlled, simulated environment	Real-world, uncontrolled environment
Timing	Before releasing to external users	After alpha testing, before final release
Visibility to Developers	Developers are present and can observe directly	Developers are typically not present; feedback is collected remotely
Advantages	Issues can be caught and fixed early in a controlled setting	Reveals real-world issues that internal testing might miss (varied environments/usage patterns)
Disadvantages	May not reveal issues that only appear in real user environments	Less controlled, feedback quality may vary, defects found later are costlier to fix

Q4. Define Formal Technical Review. Explain FTR meeting guidelines and the review reporting/record-keeping process. *[8 Marks]*

In Simple English

A Formal Technical Review (FTR) is a structured meeting where a small group of qualified people carefully examine a piece of work (like a design or code) to find mistakes before they cause bigger problems later.

Definition

A Formal Technical Review is a planned, structured meeting conducted by software engineers (and other stakeholders) to evaluate a software work product (requirements, design, code) with the aim of uncovering errors in function, logic, or implementation before it moves further in the development process.

FTR Meeting Guidelines

- Keep the review team small (typically 3-5 people) and well-prepared in advance
- Set and follow a clear agenda, focusing on the work product, not the individual who produced it
- Limit the meeting duration (commonly 2 hours or less) to maintain focus and effectiveness
- Raise issues, but avoid attempting to resolve every problem during the meeting itself — record them for later resolution
- Take written notes of all issues identified during the review
- Establish a review agenda and stick to it, avoiding scope creep into unrelated topics

Review Reporting and Record-Keeping Process

- Produce a review issues list documenting all problems identified during the meeting
- Prepare a review summary report answering: what was reviewed, who reviewed it, and what were the findings/recommendations
- Maintain a record of all reviews conducted, to track recurring issues and support process improvement
- Assign follow-up action items with responsible owners and deadlines for resolving identified issues

Q5. Explain the relationship between reliability, availability, and serviceability in the context of systems or products, and discuss whether these factors are interdependent. *[8 Marks]*

In Simple English

A system that rarely breaks (reliable), is almost always ready to use (available), and is quick and easy to fix when something does go wrong (serviceable) — these three qualities work together, and improving one often helps or requires the others.

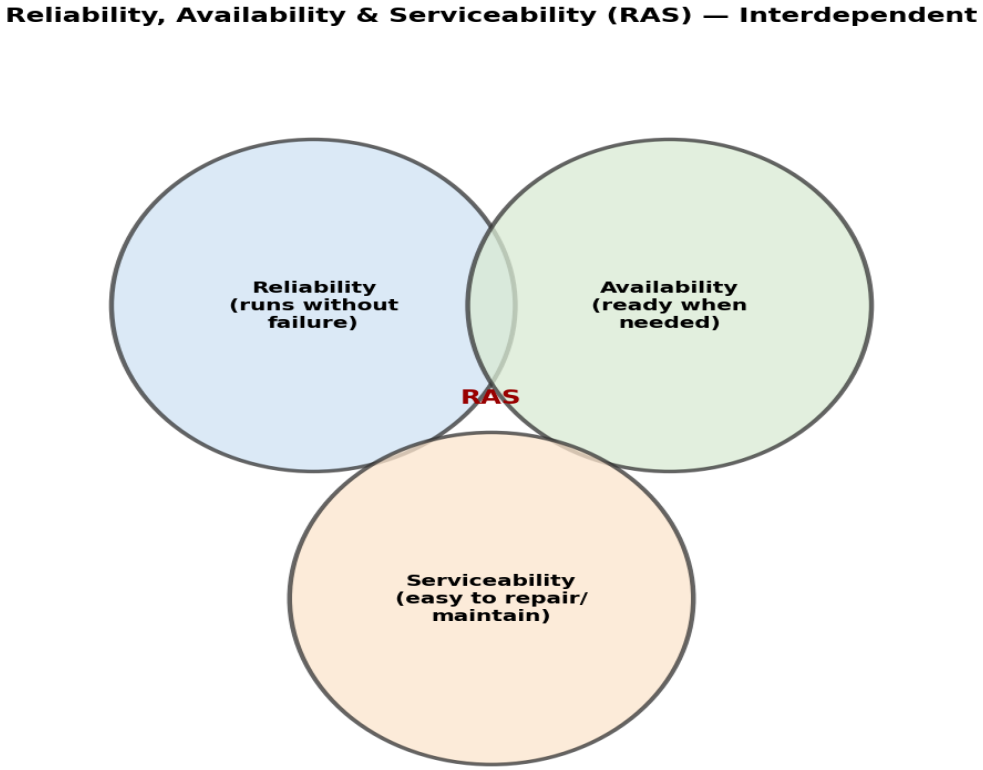

Figure: Reliability, Availability, and Serviceability (RAS)

Definitions

Reliability: The probability that a system performs its intended function correctly, without failure, over a specified period.

Availability: The proportion of time a system is operational and accessible for use when required, often expressed as: Availability = Uptime / (Uptime + Downtime).

Serviceability: The ease and speed with which a system can be repaired, maintained, or restored to operation after a failure (also called maintainability).

Are They Interdependent?

Yes — these three factors are closely interdependent:

- Higher reliability directly increases availability, since fewer failures mean less downtime
- Better serviceability (faster repair/recovery) increases availability even when failures do occur, by minimizing the downtime per failure
- A highly reliable system that is very hard to service may still have poor practical availability if rare failures take a long time to fix
- Conversely, a system that fails often but can be repaired almost instantly may still achieve high availability despite lower reliability

In practice, system designers must balance investment across all three — since maximizing reliability alone is often more expensive than achieving the same availability target through a combination of moderate reliability and excellent serviceability.

Q6. Explain the Clean Room process. Describe the Clean Room Development process and the Clean Room Certification process. *[8 Marks]*

In Simple English

Cleanroom Software Engineering is an approach that tries to prevent bugs from ever being written in the first place — using careful mathematical design and reasoning — instead of relying mainly on testing to catch bugs after the fact.

The Clean Room Process (Overview)

Cleanroom Software Engineering is a software development process that emphasizes defect prevention through rigorous, formal design and verification (using mathematical reasoning about correctness), rather than defect removal through extensive testing. Statistical usage testing (rather than exhaustive testing of all code paths) is then used to certify reliability.

Clean Room Development Process

1. Formal Specification: The software requirements are specified precisely using a formal (mathematically-based) notation.
2. Incremental Development: The system is developed in small, manageable increments, each fully specified and verified before moving to the next.
3. Box-Structure Design: Design proceeds through 'black box', 'state box', and 'clear box' representations that progressively reveal implementation detail while preserving correctness.
4. Correctness Verification: Rather than testing code by execution, teams perform formal, team-based correctness verification — mathematically proving that the code satisfies its specification.

Clean Room Certification Process

5. Statistical Use Testing: Test cases are generated based on the anticipated statistical profile of real-world usage (rather than testing every possible code path).
6. Reliability Modeling: The failure data collected during statistical testing is used to estimate the software's current reliability using statistical models.
7. Certification: Once the estimated reliability meets the target threshold, the software component is certified as ready for release/integration.

Q7. What are the concepts and elements of the object model? Explain Object-Oriented Design (OOD) and its importance in software development. *[8 Marks]*

In Simple English

The object model is the way of thinking about a system as a collection of interacting 'objects' — each bundling its own data and behavior together — rather than as a sequence of steps/functions.

Elements of the Object Model

- **Abstraction:** Focusing on the essential characteristics of an object relevant to the problem, while ignoring irrelevant detail.
- **Encapsulation:** Bundling an object's data (attributes) and behavior (methods) together, hiding internal implementation details.
- **Modularity:** Decomposing a system into a set of cohesive, loosely-coupled modules (objects/classes).
- **Hierarchy:** Organizing abstractions into ranked/ordered structures, such as inheritance (is-a) and aggregation (part-of) hierarchies.
- **Typing:** Enforcing that objects of a particular class can only be interchanged with objects of compatible types.
- **Concurrency:** Allowing different objects to operate simultaneously/independently.
- **Persistence:** An object's ability to retain its state across time and/or space (e.g., saved to a database).

Object-Oriented Design (OOD) and Its Importance

Object-Oriented Design is a design methodology that models a software system as a collection of interacting objects, each an instance of a class with defined attributes and behaviors, using concepts such as encapsulation, inheritance, and polymorphism.

- Promotes reusability, since well-designed classes can be reused across different parts of a system or in future projects
- Improves maintainability, since encapsulation isolates changes to individual objects without affecting the rest of the system
- Models real-world entities naturally, making designs more intuitive and easier to communicate with stakeholders
- Supports extensibility through inheritance and polymorphism, allowing new functionality to be added with minimal disruption to existing code

Q8. Describe the basic/current/fundamental trends in software engineering. Analyze the statement that SaaS is a significant/major trend in today's IT-related businesses and share your perspective. *[8 Marks]*

In Simple English

Software engineering keeps evolving — today's big trends include using AI to help build software, moving everything to the cloud, working in small fast iterations (Agile/DevOps), and renting software instead of buying it (SaaS).

Current Trends in Software Engineering

- **Agile and DevOps:** Emphasizing iterative development, continuous integration/delivery (CI/CD), and close collaboration between development and operations teams.
- **Cloud Computing:** Shifting infrastructure and services to cloud platforms for scalability and cost-efficiency.
- **Software as a Service (SaaS):** Delivering software as a subscription-based, centrally-hosted service rather than installed products.
- **Artificial Intelligence & Machine Learning Integration:** Using AI/ML both as features within applications and as tools to assist development (e.g., AI-assisted coding).
- **Microservices Architecture:** Decomposing large applications into small, independently deployable services.
- **Low-Code/No-Code Development:** Enabling faster application development with minimal hand-written code.
- **DevSecOps:** Integrating security practices throughout the development lifecycle rather than as an afterthought.

Analysis: Is SaaS a Major Trend?

SaaS (Software as a Service) has indeed become one of the most significant trends in modern IT businesses, for several reasons:

- Lower upfront costs for customers (subscription-based pricing vs large upfront licensing fees), making software accessible to more businesses
- Centralized updates and maintenance mean all users always run the latest, most secure version without manual upgrades
- Scalability — cloud-hosted SaaS can easily scale resources up or down based on demand
- Accessibility from any device with an internet connection, supporting remote/distributed work
- Recurring revenue models give software vendors more predictable, stable income compared to one-time license sales

Overall, SaaS has fundamentally reshaped how software is delivered, purchased, and consumed, making it a genuinely transformative and major trend rather than a passing fad — though it does introduce new challenges around data security, vendor lock-in, and dependency on continuous internet connectivity.

Questions from 2024 Examinations

Q1. How would you define Verification and Validation? Discuss briefly on White Box Testing versus Black Box Testing methodologies with examples. *[8 Marks]*

In Simple English

Verification asks 'are we building the product right?' (checking against the design/spec). Validation asks 'are we building the right product?' (checking it actually meets the user's real needs).

Definitions

Verification: The process of evaluating software at each development phase to ensure it satisfies the conditions imposed by the previous phase (i.e., conforms to its specification) — 'Are we building the product right?'

Validation: The process of evaluating the final software to ensure it complies with actual user requirements and needs — 'Are we building the right product?'

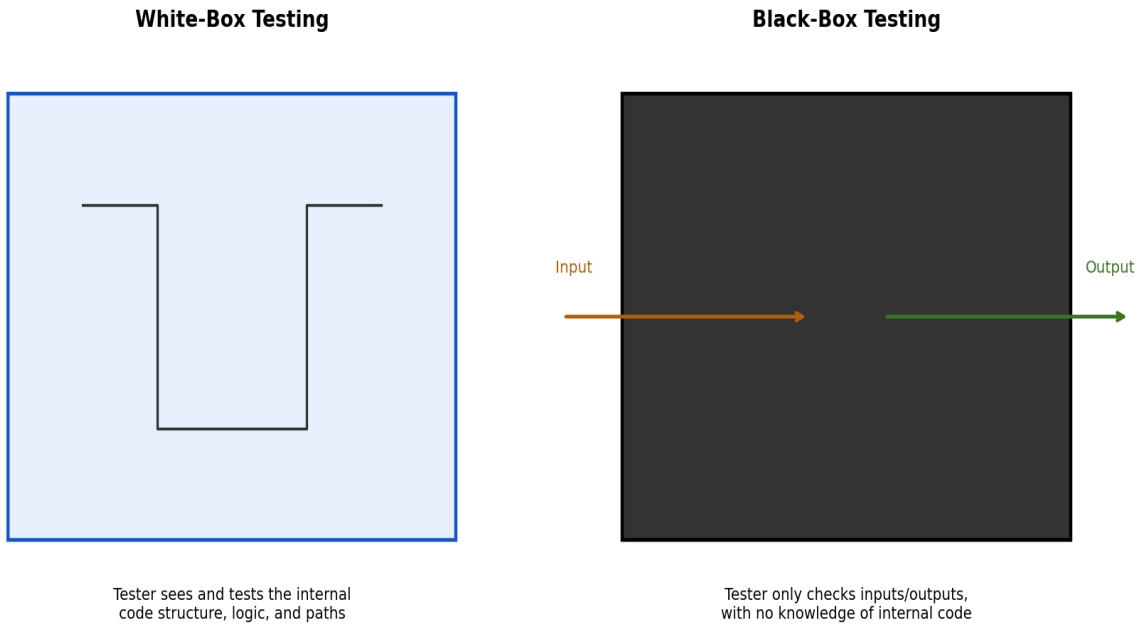

Figure: White-Box vs Black-Box Testing

White-Box Testing

Also called structural or glass-box testing, this technique tests the internal logic, code structure, and paths of the software, with the tester having full knowledge of the source code. Example: designing test cases to ensure every branch of an if-else statement is executed at least once (branch coverage).

Black-Box Testing

This technique tests the software's functionality without any knowledge of its internal code structure, focusing purely on inputs and expected outputs. Example: testing a login form by entering valid/invalid username-password combinations and checking whether the system grants or denies access appropriately, without knowing how the authentication code works internally.

Q2. What is software testing, and why is it important? Explain different types/levels of testing. What is the hierarchy of software testing, and is there any relationship with levels of testing? *[8 Marks]*

In Simple English

Software testing is the process of deliberately trying to find mistakes in software before real users do, by running it and checking whether it behaves as expected.

Definition and Importance

Software testing is the process of executing a program with the intent of finding errors, verifying that it meets specified requirements, and validating that it behaves correctly under both expected and unexpected conditions. It is important because it directly reduces the risk of costly failures, protects user trust, ensures compliance with requirements, and reduces long-term maintenance costs by catching defects early.

Levels of Testing (Hierarchy)

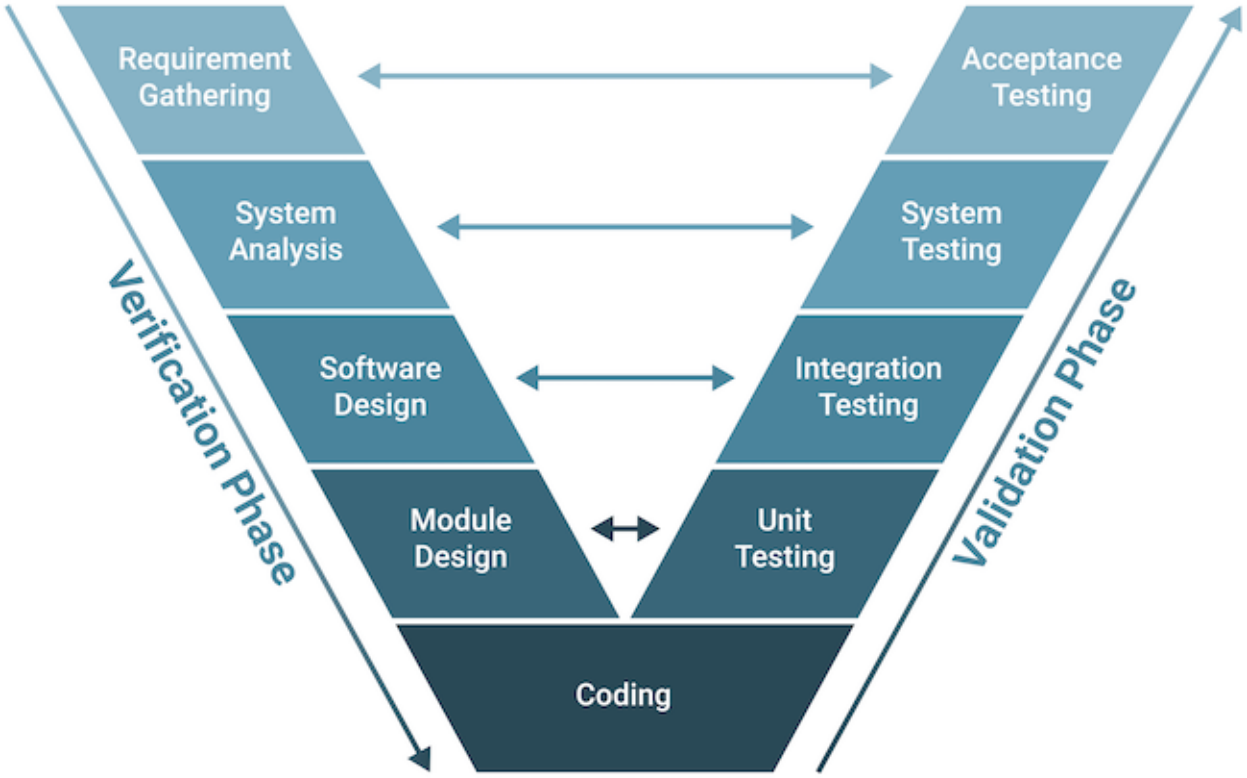

Figure: V-Model — Development Phases and Corresponding Testing Levels

1. **Requirement Gathering:** Collects and documents the business needs and user expectations for the system — corresponds to the **Acceptance Testing** phase.
2. **System Analysis:** Analyzes the initial requirements to formulate detailed system specifications and functional requirements — corresponds to the **System Testing** phase.
3. **Software Design:** Defines the high-level architecture, overall structure, and relationships between system components — corresponds to the **Integration Testing** phase.
4. **Module Design:** Breaks down the high-level design into detailed, low-level specifications for individual components — corresponds to the **Unit Testing** phase.
5. **Coding:** The actual development phase where the detailed module designs are translated into executable source code — this sits at the base of the "V" and connects the Verification and Validation phases.
6. **Unit Testing:** Tests individual components/modules in isolation, verifying they function correctly on their own — corresponds to the **Module Design** phase.
7. **Integration Testing:** Tests the interfaces and interactions between integrated units/modules — corresponds to the **Software Design** phase.
8. **System Testing:** Tests the complete, fully-integrated system against the overall system specifications — corresponds to the **System Analysis** phase.
9. **Acceptance Testing:** Validates the system against original user/business needs, ensuring it is ready for deployment — corresponds to the **Requirement Gathering** phase.

Relationship with Levels of Testing (the V-Model)

The hierarchy of testing directly mirrors the hierarchy of development phases, as depicted in the V-Model: each development phase on the left side of the V has a corresponding testing phase on the right side, verifying the work done at that stage. This ensures traceability — every requirement, design decision, and code module has a specific corresponding test level that verifies it, moving from the most detailed level (unit) up to the most abstract (acceptance).

Q3. How to measure reliability and availability of software? Explain systematically. *[8 Marks]*

In Simple English

We measure reliability by tracking how often the software fails over time, and availability by tracking what fraction of the time it's actually usable.

Measuring Reliability

Software reliability is commonly measured using metrics based on failure data collected during testing or operation:

- **Mean Time Between Failures (MTBF):** The average time elapsed between one failure and the next. MTBF = Total Operating Time / Number of Failures.
- **Mean Time To Failure (MTTF):** The average time a system operates before its first failure (used for non-repairable systems).
- **Failure Rate / Failure Intensity:** The number of failures occurring per unit of time or per unit of execution.
- **Defect Density:** The number of confirmed defects per size unit of software (e.g., per KLOC).

Measuring Availability

Availability is computed based on uptime and downtime measurements:

```
Availability = MTBF / (MTBF + MTTR)
  where MTTR = Mean Time To Repair

Or equivalently: Availability = Uptime / (Uptime + Downtime)
```

For example, if a system has an MTBF of 100 hours and an MTTR of 2 hours, its availability = 100/(100+2) ≈ 0.9804, or 98.04%. Systematically, organizations collect failure logs over the operational period, compute these metrics regularly, and track trends over time to identify whether reliability/availability is improving or degrading across software releases.

Q4. How do you define Risk? Explain in detail about risk management techniques and mitigation. *[8 Marks]*

In Simple English

A risk is something that might go wrong in the future and hurt the project if it does. Risk management is the practice of spotting these dangers ahead of time and having a plan ready, instead of just reacting in a panic when they actually happen.

Definition

A risk is a potential problem or uncertain event that, if it occurs, could negatively affect a project's schedule, budget, quality, or overall success. Risks are typically characterized by their probability of occurring and the impact/consequence if they do.

Risk Management Process

10. Risk Identification: Systematically identifying potential risks — technical, project, business, and external risks — often using checklists, brainstorming, or historical data from past projects.
11. Risk Analysis/Assessment: Estimating each risk's probability of occurrence and potential impact, often prioritizing risks using a probability-impact matrix.
12. Risk Prioritization: Ranking risks by their exposure (probability × impact) to focus mitigation efforts on the most significant threats first.
13. Risk Mitigation Planning: Developing strategies to reduce the probability or impact of high-priority risks.
14. Risk Monitoring: Continuously tracking identified risks and watching for new risks throughout the project lifecycle.

Risk Mitigation Strategies

- **Avoidance:** Changing the project plan to eliminate the risk entirely (e.g., avoiding an unproven technology).
- **Reduction/Mitigation:** Taking action to reduce the probability or impact of the risk (e.g., additional training, prototyping risky components early).
- **Transfer:** Shifting the risk to a third party (e.g., outsourcing a risky component, purchasing insurance).
- **Acceptance:** Consciously deciding to accept the risk (typically for low-probability, low-impact risks), sometimes with a contingency reserve set aside.

An effective RMMM (Risk Mitigation, Monitoring, and Management) plan documents all identified risks along with their mitigation strategies and assigns responsibility for ongoing monitoring throughout the project.

Q5. What is the object-oriented paradigm? Discuss recursive/parallel model work. *[8 Marks]*

In Simple English

The object-oriented paradigm is a way of programming and designing software around 'objects' (things that bundle data and behavior together), rather than around a sequence of instructions.

Object-Oriented Paradigm

The object-oriented paradigm is a programming and design approach based on the concept of 'objects' — instances of classes that encapsulate both data (attributes/state) and behavior (methods), interacting with each other through well-defined interfaces. Its core principles include encapsulation, inheritance, polymorphism, and abstraction, enabling modular, reusable, and maintainable software design.

Recursive/Parallel Development Model

The recursive/parallel development model (sometimes discussed in the context of object-oriented development) reflects the fact that different objects/subsystems within a large system can be designed, implemented, and tested somewhat independently and in parallel, since each object encapsulates its own data and behavior.

- **Recursive Aspect:** The same design process (analysis → design → implementation → testing) is applied recursively at different levels of abstraction — first at the system level, then applied again to each subsystem, then again to individual classes/objects.
- **Parallel Aspect:** Because objects are relatively independent, loosely-coupled units, different teams can work on different objects/subsystems concurrently, accelerating overall development compared to a strictly sequential process.

This approach takes advantage of the natural modularity of object-oriented systems — allowing large projects to be decomposed into smaller, semi-independent pieces of work that can each follow their own mini development cycle, in parallel with others, before being integrated.

Questions from 2025 Examinations (Pokhara University Spring)

Q1. Explain Agile methods and highlight how Extreme Programming (XP) fits within Agile practices. Explain the lifecycle of XP. *[8 Marks]*

In Simple English

Agile is a way of building software in small, quick steps, getting frequent feedback, instead of trying to plan everything perfectly at the start. XP is one specific, very disciplined flavor of Agile that emphasizes things like coding in pairs, testing constantly, and releasing very often.

Agile Methods

Agile methods are a family of iterative, incremental software development approaches that emphasize flexibility, customer collaboration, and rapid delivery of working software over rigid, upfront planning and documentation. Core Agile values (from the Agile Manifesto) prioritize individuals and interactions, working software, customer collaboration, and responding to change.

How XP Fits Within Agile Practices

Extreme Programming (XP) is one of the most disciplined Agile methodologies, specifically focused on improving software quality and responsiveness to changing customer requirements through very short development cycles, extensive automated testing, and close customer involvement — embodying core Agile values in concrete engineering practices.

XP Lifecycle

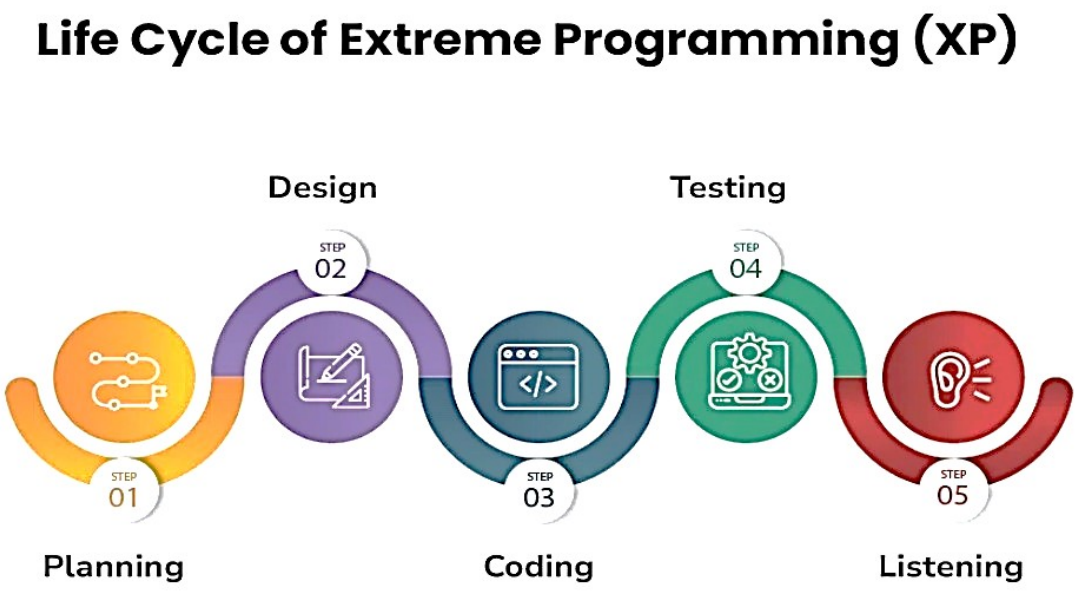

Figure: Extreme Programming (XP) Lifecycle

1. **Planning:** This is the preparation stage. You figure out exactly what you want to build, who it is for, and what resources or time you need. *(Analogy: Deciding how many rooms the house needs and determining your budget.)*
2. **Designing:** This is creating the blueprint. Before building, you decide how the software will look, how it will function, and how the different technical parts will connect. *(Analogy: Drawing up the architectural plans.)*
3. **Coding:** This is the actual construction phase. Developers write the programming code to bring the design blueprints to life and build the software. *(Analogy: Laying the bricks, doing the plumbing, and building the structure.)*
4. **Testing:** This is the inspection phase. The team carefully checks the software to make sure everything works properly and fixes any mistakes, errors, or "bugs" they find. *(Analogy: Turning on the water and electricity to make sure nothing leaks or shorts out.)*
5. **Listening:** This is the feedback phase. You show the software to the users or the client and actively listen to their opinions. Their feedback tells you what is working well and what needs to be improved in the next round of planning. *(Analogy: Asking the homeowners what they think of the house and adjusting things based on their needs.)*

Q2. Explain design patterns. Describe the Singleton, Factory, and Observer patterns and their roles with examples. *[8 Marks]*

In Simple English

Design patterns are proven, reusable solutions to common design problems — like tried-and-tested recipes that experienced developers reach for instead of reinventing a solution from scratch every time.

Design Patterns

A design pattern is a general, reusable solution to a commonly occurring problem within a given context in software design. Patterns are not finished code, but templates describing how to solve a problem that can be adapted to many different situations, improving code reusability, flexibility, and communication among developers (via a shared vocabulary).

Singleton Pattern

Ensures a class has only one instance throughout the application, and provides a single global point of access to that instance. Role: Useful for managing shared resources like a database connection pool or a configuration manager, where having multiple instances could cause conflicts or wasted resources. Example: a Logger class where all parts of an application must write to the same single log file/instance.

Factory Pattern

Provides an interface for creating objects, but lets subclasses (or a factory method) decide which specific class to instantiate, rather than the client code directly calling a constructor. Role: Decouples object creation from the code that uses the object, making it easy to introduce new types without changing existing client code. Example: a 'ShapeFactory' that returns a Circle, Square, or Triangle object based on an input parameter, without the client needing to know the exact class being instantiated.

Observer Pattern

Defines a one-to-many dependency between objects, so that when one object (the 'subject') changes state, all its dependent objects ('observers') are automatically notified and updated. Role: Useful for implementing event-handling systems and maintaining consistency between related objects without tightly coupling them. Example: in a weather monitoring app, when the WeatherStation (subject) updates its temperature reading, all registered display screens (observers) are automatically notified and refresh their displayed values.

Q3. Define encapsulation, inheritance, and polymorphism, and explain their significance in OOAD. *[8 Marks]*

In Simple English

Encapsulation means keeping an object's inner workings private and only exposing what's needed. Inheritance means a new class can reuse and extend an existing class. Polymorphism means the same action can behave differently depending on which object performs it.

Definitions

Encapsulation: The bundling of an object's data (attributes) and the methods that operate on that data into a single unit (class), while restricting direct access to some of the object's internal components (information hiding).

Inheritance: A mechanism where a new class (subclass/derived class) acquires the properties and behaviors of an existing class (superclass/base class), allowing code reuse and the modeling of hierarchical relationships.

Polymorphism: The ability of different classes/objects to respond to the same method call or message in different, class-specific ways (e.g., a 'draw()' method behaves differently for a Circle object versus a Square object).

Significance in Object-Oriented Analysis and Design (OOAD)

- **Encapsulation:** Improves maintainability and security by preventing external code from directly manipulating an object's internal state, reducing unintended side effects and making changes to internal implementation safer.
- **Inheritance:** Reduces code duplication and supports the modeling of natural hierarchies (e.g., 'Manager' inherits from 'Employee'), making designs easier to extend with new subclasses.
- **Polymorphism:** Increases flexibility and extensibility, allowing new classes to be added that work seamlessly with existing code (e.g., adding a new Shape subclass that automatically works wherever Shape objects are used), without modifying existing logic.

Q4. Why is code readability important in software development? Explain the concept of Clean Code. Suggest techniques to improve readability and explain how refactoring contributes to software development with its benefits. *[8 Marks]*

In Simple English

Code is read far more often than it's written — by other developers, and by your future self. If code is messy and confusing, even small changes become slow and risky. Clean, readable code saves everyone time and prevents bugs.

Why Code Readability Is Important

- Code is read many more times than it is written, during debugging, code review, and future feature additions
- Readable code is easier to debug and maintain, directly reducing long-term development costs
- Improves collaboration — team members can understand and safely modify each other's code
- Reduces the likelihood of introducing new bugs when making changes to poorly understood code

Clean Code

Clean Code refers to a set of principles and practices (popularized by Robert C. Martin) for writing code that is simple, readable, well-organized, and easy to understand and maintain — code that clearly expresses its intent, uses meaningful names, and avoids unnecessary complexity or duplication.

Techniques to Improve Readability

- Use meaningful, descriptive names for variables, functions, and classes
- Keep functions/methods small and focused on a single responsibility
- Maintain consistent formatting and indentation throughout the codebase
- Write clear comments only where necessary (explaining 'why', not restating 'what' the code obviously does)
- Avoid deep nesting of conditionals/loops; use early returns/guard clauses instead
- Remove dead code and unnecessary complexity

Refactoring and Its Benefits

Refactoring is the process of restructuring existing code — improving its internal structure, readability, and design — without changing its external behavior or functionality.

- Improves code readability and maintainability without altering functionality
- Reduces technical debt accumulated over time, making future changes faster and safer
- Helps identify and eliminate duplicated code, improving consistency
- Makes it easier to add new features, since a cleaner codebase is simpler to extend
- Reduces the risk of defects by simplifying overly complex logic

Q5. Why is software maintenance an essential phase in the software development life cycle? Analyze with suitable examples. *[8 Marks]*

In Simple English

Software doesn't stop needing attention once it's released — bugs are found, technology changes, and users want new features, so ongoing maintenance is what keeps software useful and working over its whole lifetime.

Why Maintenance Is Essential

- **Corrective Maintenance:** Fixes defects/bugs discovered after release that weren't caught during testing. Example: patching a security vulnerability found after deployment.
- **Adaptive Maintenance:** Modifies software to work correctly in a changing environment (new OS versions, hardware, regulations). Example: updating an app to remain compatible with a new mobile OS version.
- **Perfective Maintenance:** Enhances the software with new features or improves performance based on evolving user needs. Example: adding a dark-mode feature requested by users.
- **Preventive Maintenance:** Proactively restructures/improves code to prevent future problems (related to refactoring). Example: refactoring a module before it becomes too complex to safely modify.

Studies consistently show that maintenance consumes the majority (often 60-80%) of a software system's total lifecycle cost, since software must continuously adapt to changing requirements, environments, and discovered defects long after its initial release — making maintenance not an afterthought, but a core, essential, and ongoing phase of the SDLC.

Q6. Draw a use case diagram for a Hostel Management System and describe the use case for 'Room Allocation'. *[8 Marks]*

In Simple English

A use case diagram shows, at a glance, who (which type of user) can do what within the system — here, students, wardens, and admins each interact with the hostel system in different ways.

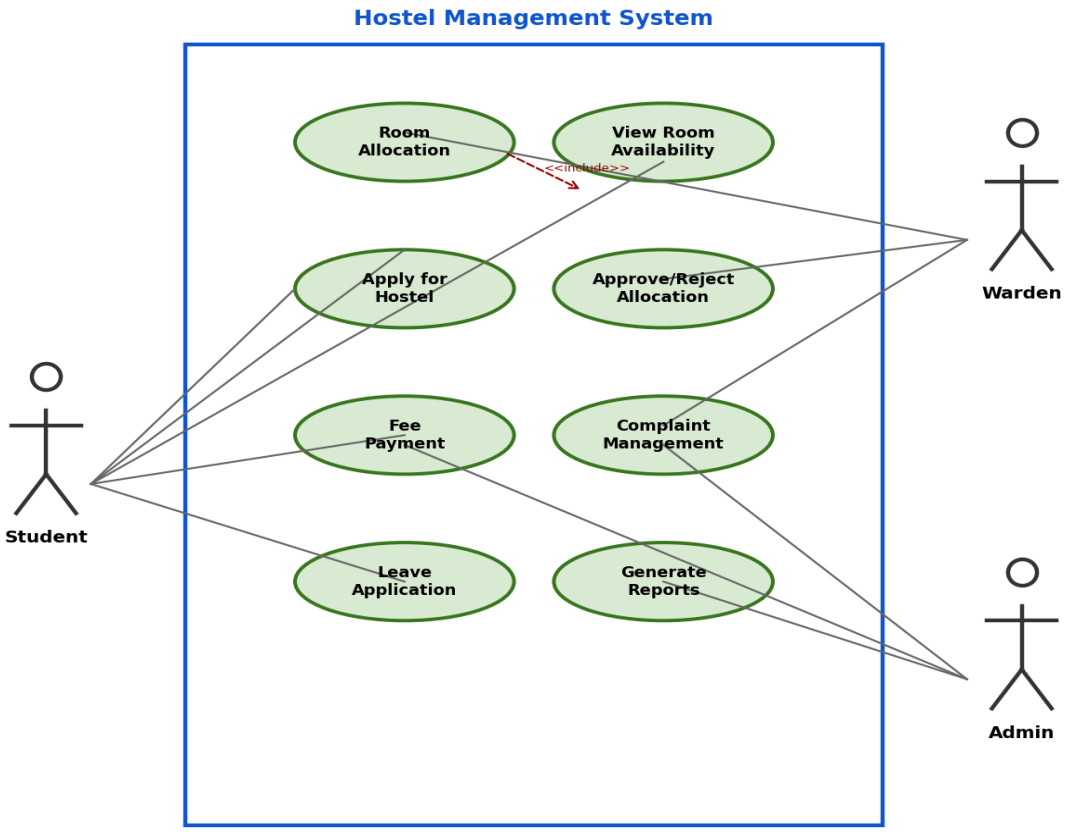

Figure: Use Case Diagram for Hostel Management System

Actors

- **Student:** Applies for hostel accommodation, pays fees, applies for leave.
- **Warden:** Approves/rejects room allocation requests, manages complaints.
- **Admin:** Oversees the overall system, generates reports, manages room inventory.

Use Case Description: Room Allocation

Field	Description
Use Case Name	Room Allocation
Actors	Student (initiator), Warden (approver)
Preconditions	Student has submitted a valid hostel application; rooms are available
Main Flow	1. Student applies for a room. 2. System checks room availability. 3. Warden reviews the application. 4. Warden approves or rejects the request. 5. If approved, system updates room occupancy and notifies the student.
Alternate Flow	If no rooms are available, the system places the student on a waiting list and notifies them.
Postconditions	The selected room's status is updated to 'occupied' and assigned to the student.

Q7. Draw and explain a sequence diagram for a Train Ticket Booking System, showing the flow from search to booking. *[8 Marks]*

In Simple English

A sequence diagram shows the exact order of messages passed between different parts of the system over time — here, from the moment a passenger searches for a train to the moment they receive a confirmed ticket.

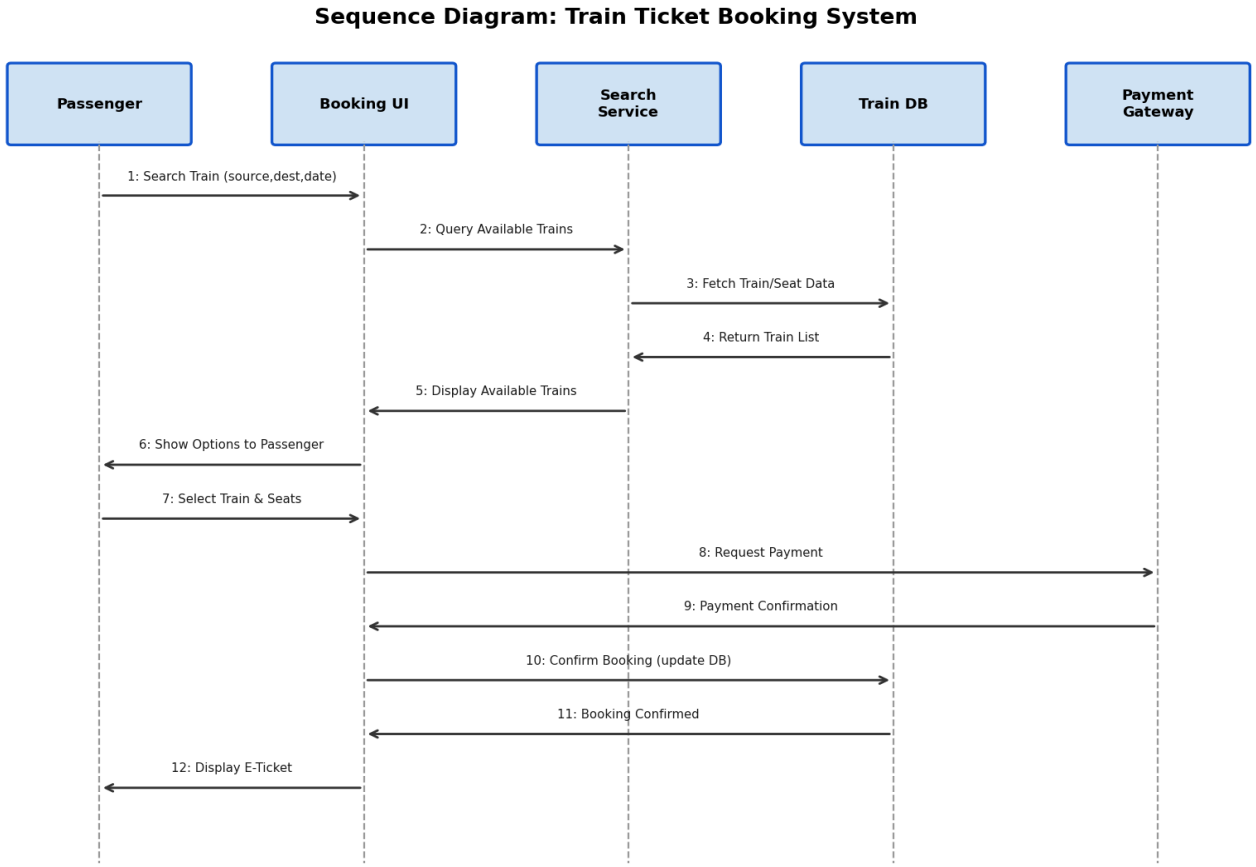

Figure: Sequence Diagram for Train Ticket Booking System

Q8. Compare and contrast the Spiral Model and the Incremental Model. Which is more adaptable to changing requirements? *[8 Marks]*

In Simple English

The Spiral model repeats a full risk-checking cycle over and over, getting more detailed each time — it's great when there's a lot of uncertainty and risk. The Incremental model just builds and delivers the product piece by piece, adding new features each round.

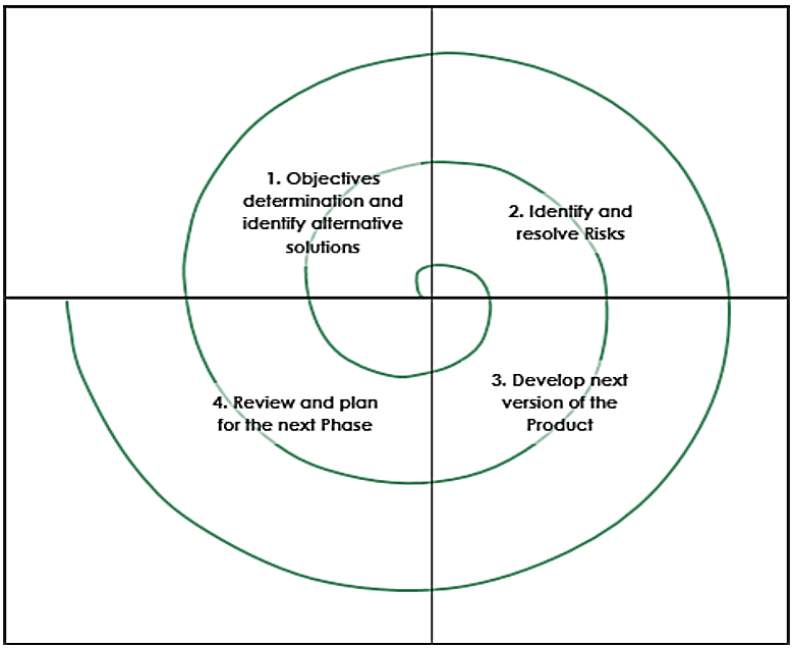

Figure: Spiral Model

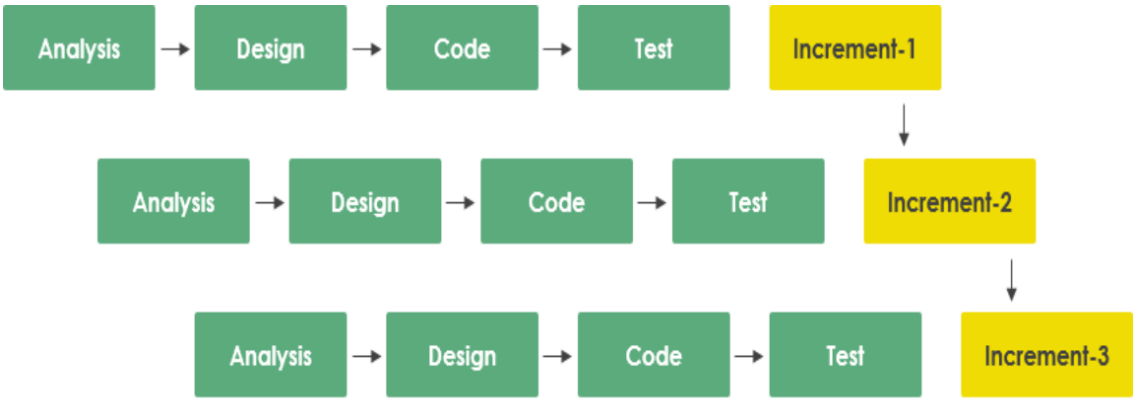

Figure: Incremental Model

Aspect	Spiral Model	Incremental Model
Core Idea	Repeated cycles (spirals) through planning, risk analysis, engineering, and evaluation, growing in detail each time	The system is built and delivered in a series of increments, each adding new functionality to the previous one
Risk Management	Explicit, dedicated risk analysis phase in every cycle — designed specifically for high-risk projects	No dedicated risk-analysis phase; risk is managed only implicitly through incremental delivery
Best Suited For	Large, complex, high-risk projects with significant uncertainty	Projects where requirements are reasonably well understood but can be delivered in usable pieces
Flexibility to Change	High — each cycle can incorporate new risk assessments and requirement changes	Moderate — changes can be incorporated in future increments, but not through formal risk-driven re-planning
Cost	Can be expensive due to repeated risk analysis	Generally more cost-predictable, as increments are smaller and well-defined

Which Is More Adaptable to Changing Requirements?

The Spiral Model is generally more adaptable to changing requirements, because each cycle of the spiral explicitly revisits objectives, risks, and alternatives before proceeding — allowing requirement changes and newly discovered risks to be formally re-evaluated and incorporated at every iteration. The Incremental Model can also accommodate change between increments, but it lacks the Spiral Model's built-in, systematic risk-and-requirement reassessment process, making it comparatively less flexible when requirements are highly volatile or poorly understood at the outset.

Q9. What is software re-engineering? Explain how version control systems (like Git) help in managing software evolution. *[8 Marks]*

In Simple English

Re-engineering means taking old software and rebuilding/improving its internal structure without changing what it does for the user — like renovating a building's interior. Version control (like Git) is the tool that keeps track of every change made to code over time, so nothing is ever lost and multiple people can work together safely.

Software Re-Engineering

Software re-engineering is the examination and alteration of an existing software system to reconstitute it in a new, improved form, and the subsequent implementation of that new form — typically including reverse engineering (understanding the existing system), restructuring, and forward engineering (rebuilding it with improved structure), without changing its overall external functionality.

Role of Version Control Systems (e.g., Git) in Managing Software Evolution

- **Change History Tracking:** Every modification to the codebase is recorded with details of what changed, when, and by whom — providing a complete audit trail of the software's evolution.
- **Branching and Merging:** Developers can work on new features or fixes in isolated branches without disrupting the main codebase, then merge changes back safely once verified.
- **Collaboration:** Multiple developers can work simultaneously on the same codebase without overwriting each other's work.
- **Rollback Capability:** If a change introduces a defect, teams can revert to a previous known-good version quickly.
- **Release Management:** Tags and branches allow teams to manage multiple versions/releases of software simultaneously (e.g., maintaining an older stable release while developing new features).

Q10. Define Software. Highlight the key differences between Software Engineering and System Engineering. *[8 Marks]*

In Simple English

Software is the set of programs, data, and documentation that make a computer do useful things. Software Engineering focuses specifically on building that software; System Engineering is the broader discipline of building a whole system — hardware, software, and people working together.

Definition of Software

Software is a collection of computer programs, procedures, associated documentation, and data that together perform a specified set of functions, providing the desired features, performance, and reliability required by the user.

Software Engineering vs System Engineering

Aspect	Software Engineering	System Engineering
Scope	Focused specifically on the design, development, and maintenance of software products	Broader discipline covering the design and management of complex systems, including hardware, software, people, and processes
Focus	Software components, code quality, software architecture	Overall system integration, interfaces between subsystems (hardware/software/human)
Deliverable	A software application/product	A complete, integrated system (which may include software as one component)
Example	Developing a mobile banking application	Designing an entire air-traffic control system, including radar hardware, software, and human operator procedures

Q11. Explain how UML diagrams such as Use Case diagram, State Diagram, Activity diagram, and Data Flow Diagrams (DFD) can be used to represent software behavior with examples. *[8 Marks]*

In Simple English

Different UML diagrams are like different camera angles on the same system — each one highlights a different aspect of how the software behaves and is structured.

- **Use Case Diagram:** Shows the functional behavior of a system from the user's perspective — which actors interact with which system functions. Example: a 'Library System' use case diagram showing a Member can 'Search Book' and 'Borrow Book', while a Librarian can 'Issue Book' and 'Manage Catalogue'.
- **State Diagram (State Machine Diagram):** Shows the different states an object can be in over its lifetime, and the events/transitions that move it between states. Example: an 'Order' object moving through states Placed → Confirmed → Shipped → Delivered, with each transition triggered by a specific event.
- **Activity Diagram:** Models the flow of control/activities within a process or business workflow, similar to a flowchart, including decision points and parallel activities. Example: an activity diagram for 'Online Order Processing' showing steps from placing an order, checking inventory, processing payment, and shipping — with decision branches for out-of-stock items.
- **Data Flow Diagram (DFD):** Represents how data moves through a system, between external entities, processes, and data stores, without showing control logic/timing. Example: a level-1 DFD for a 'Payroll System' showing data flowing from the 'Employee' entity into a 'Calculate Salary' process, which reads from an 'Employee Records' data store and outputs a 'Payslip'.

Together, these diagrams provide complementary views: use case diagrams capture functional scope, state diagrams capture object lifecycle behavior, activity diagrams capture process/workflow logic, and DFDs capture data movement — giving a complete picture of software behavior from multiple perspectives.

Q12. How does the review method help reduce errors? Please explain defect amplification and removal with a diagram. *[8 Marks]*

In Simple English

Reviews are a way of catching mistakes by having other people look carefully at your work before it moves forward — catching problems early stops them from multiplying into bigger, more expensive problems down the line.

How Reviews Help Reduce Errors

- Reviews (formal technical reviews, walkthroughs, inspections) catch defects at the same phase they were introduced, before they can propagate downstream
- Multiple reviewers bring different perspectives, catching mistakes a single author might overlook
- Encourages adherence to standards and best practices, reducing the introduction of new defects
- Provides early feedback, making corrections far cheaper than fixing the same defect after it has propagated through several phases

Defect Amplification and Removal (with Diagram)

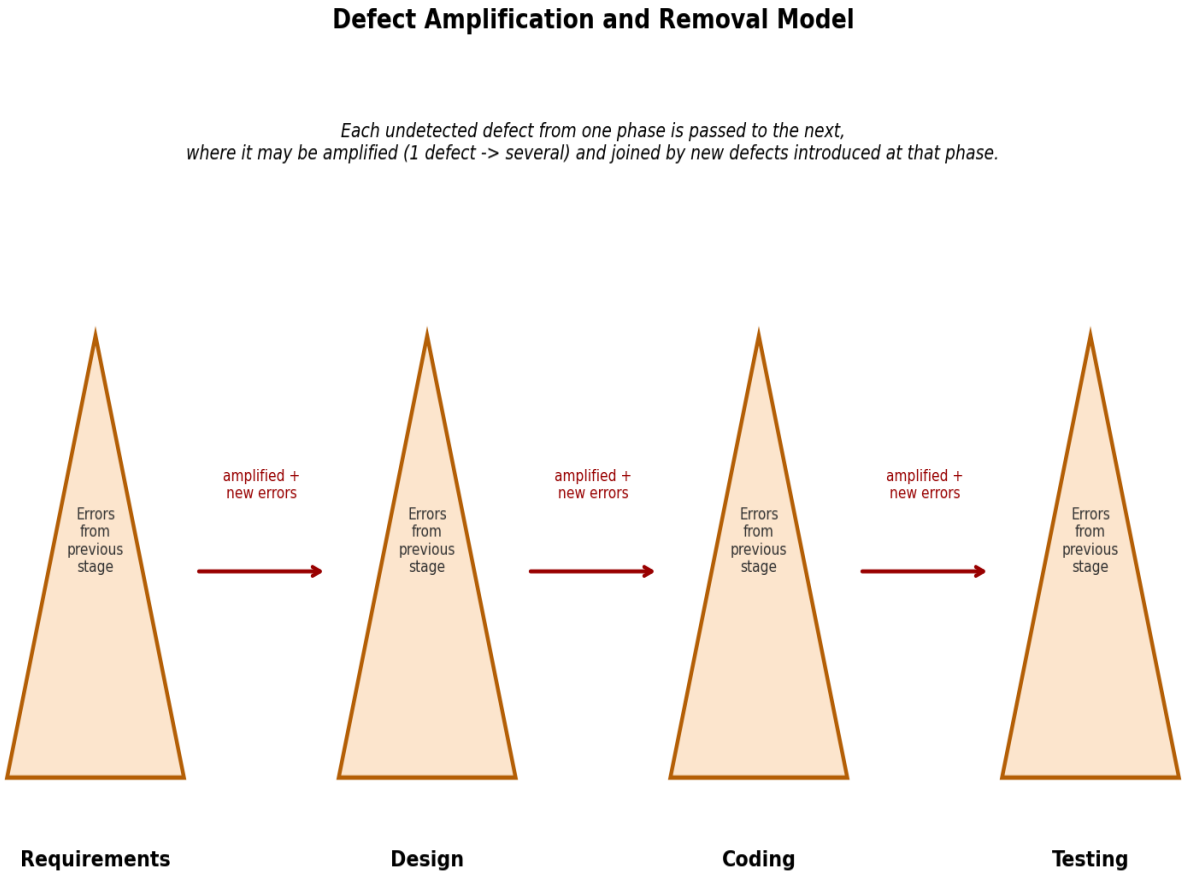

Figure: Defect Amplification and Removal Model

The defect amplification model shows that errors from one phase (e.g., 1 requirements error) can multiply as they pass into subsequent phases (e.g., producing 2-3 design errors), and further multiply again in coding, if not caught and

removed at the phase in which they originate. Reviews conducted at each phase act as a 'filter', removing a percentage of defects before they can be amplified further — the earlier and more effective these review filters are, the fewer defects reach final testing or production, directly reducing the total cost and effort of quality assurance.

Q13. Explain the OOP concept in Project Management. *[8 Marks]*

In Simple English

The same ideas that make object-oriented programming powerful — breaking things into self-contained pieces, reusing what already works, and organizing things into hierarchies — can also be applied to how a project itself is organized and managed.

Technical Explanation

- **Modularity in Project Structure:** Just as OOP decomposes a system into classes/objects, project management can decompose a project into independent work packages/modules (similar to a Work Breakdown Structure), each with clear ownership and interfaces — mirroring encapsulation.
- **Reusability:** Reusable project templates, checklists, and processes (analogous to reusable classes) can be applied across multiple projects, saving planning effort.
- **Hierarchy:** Project structures often mirror inheritance/aggregation hierarchies — a program consists of projects, which consist of phases, which consist of tasks — similar to how objects can be composed of other objects.
- **Encapsulation of Responsibility:** Each team/module in a project is given ownership of its own deliverables and internal processes, with only well-defined interfaces (deliverables, APIs, hand-off points) exposed to other teams — reducing unwanted dependencies, just as encapsulation reduces unwanted coupling in code.

Applying these object-oriented principles to project management encourages more modular, reusable, and maintainable project structures, particularly valuable in large, object-oriented software development projects where the project's organizational structure often naturally mirrors the software's architectural structure.

Q14. What do you understand by 'proof of correctness' in Cleanroom Software Engineering? How does it differ from traditional defect detection techniques? *[8 Marks]*

In Simple English

Instead of running the code to see if it seems to work (testing), proof of correctness means mathematically proving, on paper, that the code logically must produce the correct result for every possible case.

Proof of Correctness

In Cleanroom Software Engineering, proof of correctness is a formal, team-based verification technique where developers mathematically demonstrate that a piece of code correctly implements its formal specification, using structured logical arguments (often based on function-theoretic reasoning about each program construct — sequence, condition, and iteration) — rather than relying on executing the code with sample inputs.

How It Differs from Traditional Defect Detection

Aspect	Proof of Correctness (Cleanroom)	Traditional Testing
Approach	Mathematical/logical verification of code against its specification	Executing the code with sample inputs and observing actual outputs
Coverage	Can, in principle, verify correctness for all possible inputs	Can only verify the specific inputs/cases actually tested
When Applied	Applied during/after coding, before any execution	Applied after code is written and can be executed
Goal	Defect prevention — proving correctness before execution	Defect detection — finding defects that already exist through execution
Effort	Requires significant upfront mathematical/logical rigor	Requires designing and executing comprehensive test cases

Cleanroom's proof-of-correctness approach aims to prevent defects from ever being introduced (a defect-prevention philosophy), whereas traditional testing is fundamentally a defect-detection philosophy — finding bugs after the code already exists, by exercising it with representative inputs.

Q15. Why do we need software configuration management? Explain its activities. *[8 Marks]*

In Simple English

As software is built, many things change constantly — code, documents, requirements — and different versions exist at once. Software Configuration Management (SCM) is the discipline of carefully tracking and controlling all these changes so nothing gets lost or confused.

Why We Need SCM

- Software undergoes continuous change throughout development and maintenance — SCM ensures these changes are controlled and tracked systematically
- Prevents confusion when multiple developers work on the same codebase simultaneously
- Enables reproducibility — the exact state of the software at any past point (e.g., a specific released version) can always be recreated
- Supports traceability between requirements, code, and test cases across versions
- Reduces risk when integrating changes from multiple team members or branches

SCM Activities

- **Identification:** Identifying and naming configuration items (source code, documents, test cases) that need to be tracked and controlled.
- **Version Control:** Managing multiple versions/revisions of each configuration item over time (often using tools like Git).
- **Change Control:** Formally reviewing, approving, and tracking proposed changes to configuration items through a defined change-request process.
- **Configuration Auditing:** Verifying that the software product matches its intended configuration and that all changes have been properly recorded and approved.
- **Status Accounting:** Recording and reporting the status of configuration items and change requests throughout the project.

Questions from 2025 Examinations (First Term & Pre-University)

Q1. Define CBSE. Explain its Phases and Components. *[8 Marks]*

In Simple English

CBSE means building software mainly by plugging together pre-built, reusable blocks (components) — similar to building something out of pre-made Lego pieces instead of molding every single brick from scratch.

Definition

Component-Based Software Engineering (CBSE) is a software development approach that emphasizes the design and construction of software systems using pre-built, reusable software components, which are then assembled and integrated to build the final application, rather than developing every part from scratch.

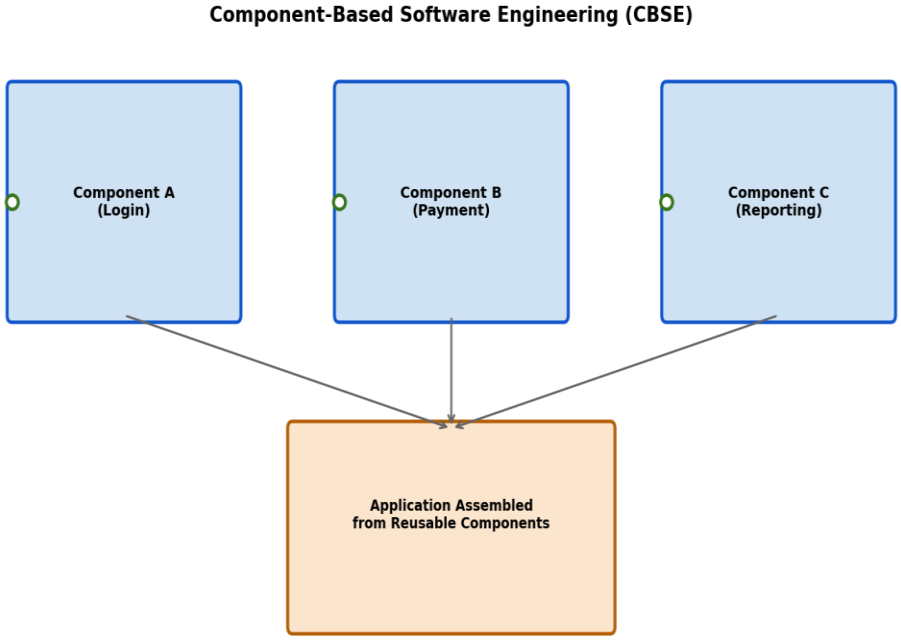

Figure: Component-Based Software Engineering

Phases of CBSE

6. Component Qualification: Evaluating candidate components (in-house or third-party) to determine whether they meet the required functional and quality needs.
7. Component Adaptation: Modifying or wrapping components (if needed) so they conform to the architecture of the system being built.
8. Component Composition/Assembly: Integrating the qualified and adapted components together, using defined interfaces, to construct the overall system.
9. Component Update/Evolution: Managing updates to components over time as requirements or underlying technologies evolve.

Components of CBSE

- **Reusable Components:** Self-contained, independently deployable units of functionality with well-defined interfaces (e.g., a payment-processing component).

- **Component Repository:** A library/catalog of available components that can be searched and selected for reuse.
- **Interfaces:** Well-defined contracts specifying how components communicate with each other and with the rest of the system.
- **Middleware/Component Framework:** Infrastructure (e.g., CORBA, COM/DCOM, EJB) that enables components to interact regardless of the platform/language they were built in.

Q2. Explain CASE Tool with its Types. *[8 Marks]*

In Simple English

CASE tools are software programs that help software engineers do their job faster and more accurately — like diagramming tools, code generators, and automated testing tools, instead of doing everything by hand.

Definition

Computer-Aided Software Engineering (CASE) tools are software applications that provide automated support for software development activities, including analysis, design, coding, testing, and project management, improving productivity and quality throughout the software development lifecycle.

Types of CASE Tools

- **Upper CASE Tools:** Support the early stages of development — requirements analysis and system design (e.g., diagramming tools for creating ER diagrams, DFDs, UML models).
- **Lower CASE Tools:** Support later stages — code generation, testing, debugging, and maintenance (e.g., automated code generators, debuggers, test-automation frameworks).
- **Integrated CASE (I-CASE) Tools:** Combine both upper and lower CASE capabilities into a single integrated environment, supporting the entire software lifecycle from analysis through maintenance.
- **Project Management CASE Tools:** Support planning, scheduling, resource allocation, and tracking of the overall project (e.g., Gantt chart tools).

Q3. Define SDLC with its Phases in detail. Compare and contrast two SDLC models / List out and explain any two types of SDLC methods. *[8 Marks]*

In Simple English

SDLC is the standard set of steps followed to build software from start to finish — understanding what's needed, designing it, building it, testing it, releasing it, and maintaining it.

Definition

The Software Development Life Cycle (SDLC) is a structured process that defines the phases involved in developing software, from initial idea/requirement gathering through design, implementation, testing, deployment, and maintenance, providing a framework to plan and control the development process.

Phases of SDLC

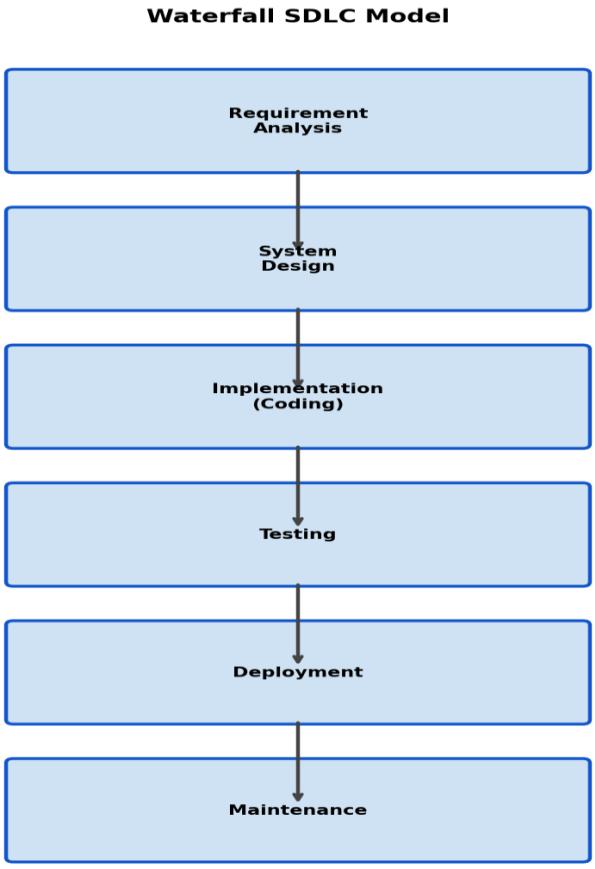

Figure: SDLC Phases (Waterfall representation)

1. Requirement Analysis: Gathering and documenting what the system must do, through discussions with stakeholders.
2. System Design: Translating requirements into a technical blueprint — architecture, data models, interfaces.
3. Implementation (Coding): Writing the actual source code based on the design specifications.
4. Testing: Verifying the software works correctly and meets requirements, at unit, integration, system, and acceptance levels.
5. Deployment: Releasing the software into the production environment for actual use.

6. Maintenance: Ongoing support, bug fixes, and enhancements after release.

Comparison of Two SDLC Models: Waterfall vs Agile

Aspect	Waterfall Model	Agile Model
Approach	Sequential — each phase must complete before the next begins	Iterative and incremental — small cycles delivering working software repeatedly
Flexibility to Change	Low — changes are difficult and costly once a phase is complete	High — designed to accommodate changing requirements at every iteration
Customer Involvement	Mainly at the start (requirements) and end (delivery)	Continuous throughout development
Documentation	Heavy, detailed documentation at each phase	Lighter documentation, prioritizing working software
Best Suited For	Projects with clear, stable, well-understood requirements	Projects with evolving or unclear requirements, needing frequent feedback

Q4. Define System Analysis and Design with its Key Aspects. *[8 Marks]*

In Simple English

System analysis is figuring out exactly what a system needs to do and why; system design is figuring out exactly how it will do it.

Definition

System Analysis: The process of studying a business or system's current situation, problems, and requirements, in order to determine what a proposed system must accomplish — 'what' needs to be built.

System Design: The process of defining the architecture, components, modules, interfaces, and data for a system to satisfy the specified requirements — 'how' it will be built.

Key Aspects

- **Requirements Gathering & Feasibility Study:** Understanding stakeholder needs and assessing technical, economic, and operational feasibility.
- **Process/Data Modeling:** Representing system processes (e.g., DFDs) and data structures (e.g., ER diagrams) clearly.
- **Architectural Design:** Defining the overall structure of the system — modules, layers, and their interactions.
- **Interface Design:** Designing how the system interacts with users and other systems.
- **Database Design:** Structuring how data will be stored, organized, and accessed.

Q5. Define Object-Oriented Analysis (OOA) and Object-Oriented Analysis and Design (OOAD). Discuss the advantages of OOAD, describe the core principles of OOP, and explain the OOD Pyramid with OOAD Objectives. *[8 Marks]*

In Simple English

OOA is about identifying the real-world 'objects' involved in a problem (like Customer, Order, Product). OOAD extends this into actually designing how those objects will be structured and interact in the final software.

Definitions

Object-Oriented Analysis (OOA): A software engineering approach that models a system's requirements by identifying and defining the real-world objects, their attributes, and their relationships relevant to the problem domain.

Object-Oriented Analysis and Design (OOAD): An extension of OOA that carries the identified objects further into a detailed technical design — defining classes, relationships (inheritance, association), interfaces, and interactions needed to implement the system.

Advantages of OOAD

- Produces designs that closely mirror real-world entities, making systems easier to understand and communicate
- Encourages reusability of classes across projects, reducing development effort
- Improves maintainability due to encapsulation, localizing the impact of changes
- Supports extensibility through inheritance and polymorphism, easing the addition of new features

Core Principles of Object-Oriented Programming (OOP)

- **Encapsulation:** Bundling data and methods together, hiding internal details.
- **Inheritance:** Deriving new classes from existing ones, reusing their properties/behavior.
- **Polymorphism:** Allowing objects of different classes to respond differently to the same message/method call.
- **Abstraction:** Modeling only the essential characteristics of an object relevant to the current context.

The OOD Pyramid and OOAD Objectives

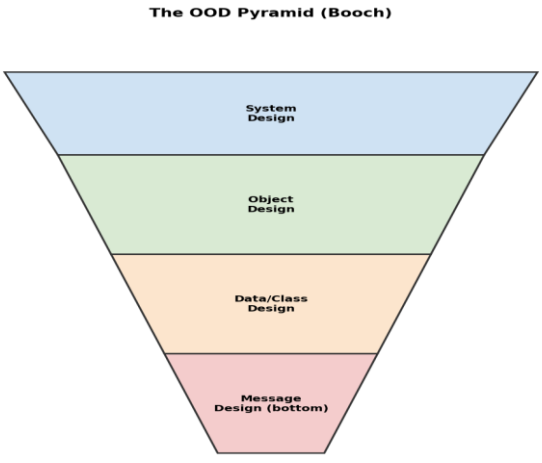

Figure: The OOD Pyramid

The OOD Pyramid (proposed by Grady Booch) represents object-oriented design across four interconnected layers, from the top (broadest) to the bottom (most detailed):

- **System Design (top):** The overall architecture of the complete system.
- **Object Design:** Defining individual objects, their responsibilities, and collaborations.
- **Data/Class Design:** Defining the internal structure (attributes, data types) of each class.
- **Message Design (bottom):** Defining the specific protocols/messages used for objects to communicate.

The overarching objective of OOAD, reflected in this pyramid, is to progressively refine a system from a high-level architectural view down to precise, implementable class and message-level detail, while preserving consistency and traceability between all four levels.

Q6. What is a Use Case Diagram? Illustrate its application by creating one for a Library Management System. *[8 Marks]*

In Simple English

A use case diagram is a simple picture showing which types of users can perform which actions in a system — giving a quick, high-level overview of what the system lets people do.

Definition

A Use Case Diagram is a UML behavioral diagram that visually represents the functional requirements of a system by showing the interactions between external actors (users or other systems) and the use cases (specific functions/services) the system provides.

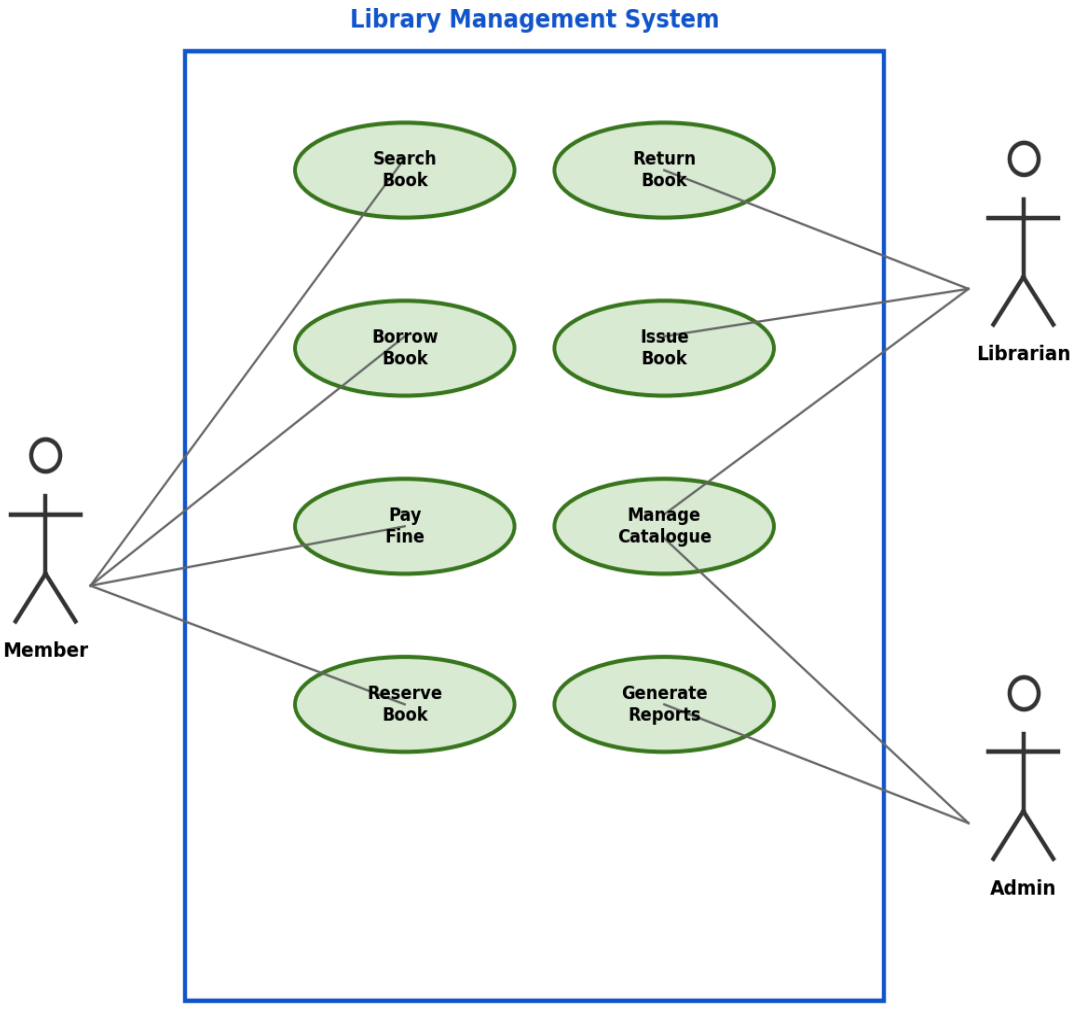

Figure: Use Case Diagram for Library Management System

Explanation

- **Member (Actor):** Can Search for Books, Borrow Books, Pay Fines, and Reserve Books.
- **Librarian (Actor):** Handles Issue Book and Return Book use cases, and Manages the Catalogue.
- **Admin (Actor):** Oversees the system through the Generate Reports use case, and can also perform catalogue management.

This diagram gives stakeholders (even non-technical ones) a quick, intuitive overview of exactly what functionality the Library Management System provides and which type of user is responsible for triggering each function.

Short Notes

The exam typically asks students to attempt a limited number of these short notes. Since a complete answer set is needed, a concise note is provided below for every topic listed.

1. RMMM Plan *[4 Marks]*

In Simple English

An RMMM plan is a document that lists all the things that could go wrong with a project and exactly what the team will do about each one.

Technical Explanation

RMMM stands for Risk Mitigation, Monitoring, and Management. It is a documented plan that records all identified project risks along with strategies to reduce their probability/impact (mitigation), methods to track those risks throughout the project (monitoring), and the overall process for handling risks if they occur (management). It ensures risk management is proactive and systematically tracked, rather than handled reactively when problems arise.

---

2. Human Computer Interaction (HCI) *[4 Marks]*

In Simple English

HCI is the study of how people use computers and software, focused on making that experience as easy, efficient, and pleasant as possible.

Technical Explanation

Human-Computer Interaction (HCI) is a multidisciplinary field concerned with the design, evaluation, and implementation of interactive computing systems for human use, focusing on usability, accessibility, and user experience. It draws on psychology, design, and computer science to ensure interfaces are intuitive, efficient, and satisfying for users.

---

3. Class Diagram *[4 Marks]*

In Simple English

A class diagram is a picture showing the main 'building blocks' (classes) of a system, what data and actions each one has, and how they're connected to each other.

Technical Explanation

A UML Class Diagram is a structural diagram that depicts the classes in a system, their attributes, methods (operations), and the relationships (association, aggregation, composition, inheritance) between them. It is one of the most widely used UML diagrams for object-oriented system design.

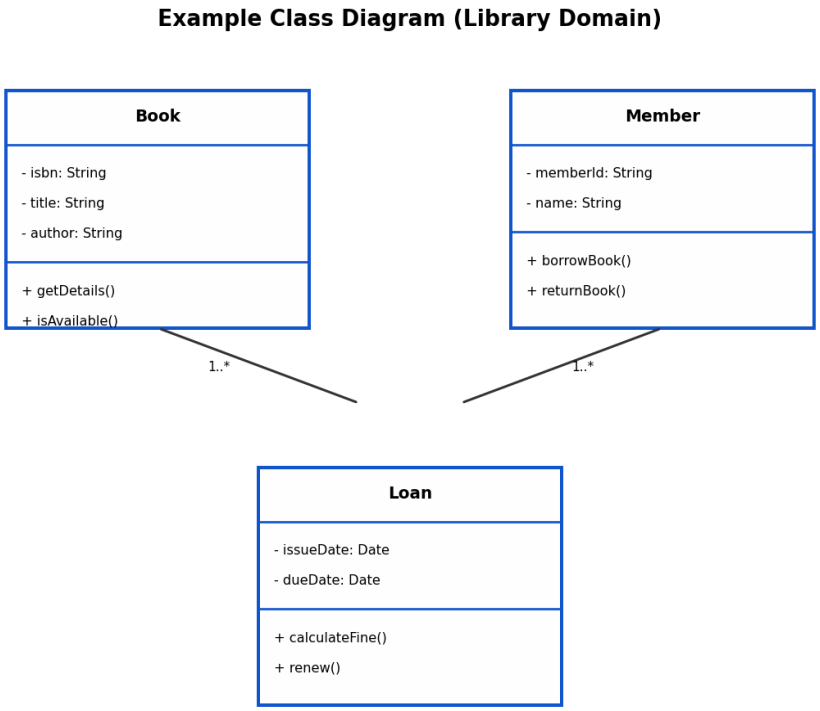

Figure: Example Class Diagram (Library Domain)

---

4. CORBA *[4 Marks]*

In Simple English

CORBA is a standard that lets pieces of software written in different languages, running on different computers, talk to each other smoothly.

Technical Explanation

CORBA (Common Object Request Broker Architecture) is a middleware standard defined by the Object Management Group (OMG) that enables software components written in different programming languages and running on different platforms/machines to communicate with one another, using an Object Request Broker (ORB) to handle the details of remote communication transparently.

5. Static vs Dynamic Testing *[4 Marks]*

In Simple English

Static testing examines the code or documents without actually running the program (like proofreading). Dynamic testing actually executes the program to see how it behaves.

Aspect	Static Testing	Dynamic Testing
Execution	Does not execute the code	Executes the code
Techniques	Reviews, walkthroughs, inspections, static analysis tools	Unit testing, integration testing, system testing
When Performed	Early in development (requirements, design, code review)	After code is written and can be run
Finds	Structural issues, standard violations, potential defects	Actual runtime behavior, functional defects

---

6. Software Safety *[4 Marks]*

In Simple English

Software safety is about making sure software doesn't cause dangerous, harmful, or life-threatening situations — this matters a lot in things like medical devices or airplane control systems.

Technical Explanation

Software safety is a software quality assurance activity focused on identifying, analyzing, and mitigating hazards that could result from software failures, particularly in safety-critical systems (e.g., medical devices, aviation, nuclear control systems), where a defect could lead to injury, loss of life, or significant environmental/financial damage. It typically involves hazard analysis, formal verification, and rigorous testing beyond standard quality assurance practices.

---

7. Line of Code (LOC) *[4 Marks]*

In Simple English

LOC is simply a count of how many lines of code a program has — used as a rough way to measure the size of a software project.

Technical Explanation

Lines of Code (LOC) is a size-oriented software metric that measures the size of a program by counting the number of lines in its source code (often excluding blank lines and comments). It is commonly used as an input to effort-estimation models like COCOMO (expressed as KLOC — thousands of lines of code), although it has limitations since it doesn't account for code complexity or the productivity differences between programming languages.

8. Software Engineering Ethics *[4 Marks]*

In Simple English

This refers to the moral responsibilities software engineers have — being honest, protecting user privacy, and not building software that harms people — beyond just making it technically work.

Technical Explanation

Software Engineering Ethics refers to the professional code of conduct and moral responsibilities that guide software engineers in their work, including honesty in reporting progress/limitations, respecting user privacy and data protection, avoiding conflicts of interest, ensuring software safety and quality, and taking responsibility for the societal impact of the systems they build. Professional bodies such as the ACM and IEEE publish formal codes of ethics for software professionals.

---

9. Testing Strategies *[4 Marks]*

In Simple English

A testing strategy is the overall game plan for how and when different types of testing will be carried out across a project, rather than just individual test cases.

Technical Explanation

A software testing strategy is a systematic roadmap that integrates test planning, test case design, execution, and result evaluation across all levels of testing (unit, integration, system, acceptance), typically following a structured progression from testing small components in isolation to validating the complete system against user requirements — often visualized using the V-Model, which pairs each development phase with its corresponding testing level.

---

10. Rational Unified Process (RUP) *[4 Marks]*

In Simple English

RUP is a structured, step-by-step software development framework, organized into repeated cycles with four major phases, that combines iterative development with strong emphasis on use cases and architecture.

Technical Explanation

The Rational Unified Process (RUP) is an iterative software development process framework developed by Rational Software (now IBM), organized around four sequential phases — Inception, Elaboration, Construction, and Transition — each consisting of one or more iterations, and structured around key disciplines like requirements, analysis & design, implementation, and testing. RUP is use-case driven, architecture-centric, and emphasizes iterative and incremental development throughout.

11. Requirement Modelling in OOAD *[4 Marks]*

In Simple English

This means using diagrams and models (rather than just plain text) to represent what a system must do, from an object-oriented perspective.

Technical Explanation

Requirement modelling in OOAD involves representing a system's functional and behavioral requirements using object-oriented models such as Use Case Diagrams (capturing functional requirements from an actor's perspective), Class Diagrams (capturing key domain entities), Activity Diagrams (capturing workflows), and Sequence Diagrams (capturing interaction between objects over time). These models provide a more precise, visual, and testable representation of requirements than plain natural-language descriptions alone.

---

12. Software Maintenance *[4 Marks]*

In Simple English

Software maintenance is everything that happens to software after it's released — fixing bugs, adapting to new environments, and adding requested improvements.

Technical Explanation

Software maintenance is the modification of a software product after delivery, to correct defects (corrective), adapt it to a changed environment (adaptive), improve performance/maintainability (perfective), or prevent future problems (preventive). It is often the longest and most costly phase of the software lifecycle, since software must continually evolve to remain useful.

---

13. Software Risk and Its Mitigation *[4 Marks]*

In Simple English

A software risk is anything that could go wrong during a project; mitigation is the plan for reducing the chance or impact of that risk.

Technical Explanation

A software risk is an uncertain event or condition that, if it occurs, has a negative effect on a project's objectives (schedule, cost, scope, or quality). Common categories include project risks (schedule/resource issues), technical risks (design/implementation problems), and business risks (market or organizational issues). Mitigation strategies include risk avoidance, risk reduction, risk transfer (e.g., outsourcing or insurance), and risk acceptance — often documented formally within an RMMM plan.

14. COCOMO Model *[4 Marks]*

In Simple English

COCOMO is a formula-based method to estimate how much effort and time a software project will need, based mainly on its estimated size.

Technical Explanation

The Constructive Cost Model (COCOMO), developed by Barry Boehm, estimates software development effort (in person-months) and schedule (in months) as a function of estimated project size (in KLOC), using empirically-derived constants that vary based on project complexity: Organic (simple), Semi-Detached (medium), or Embedded (complex, tightly constrained). The Basic COCOMO formula is Effort = a × (KLOC)^b, and Development Time = c × (Effort)^d, with different constants a, b, c, d for each project mode.

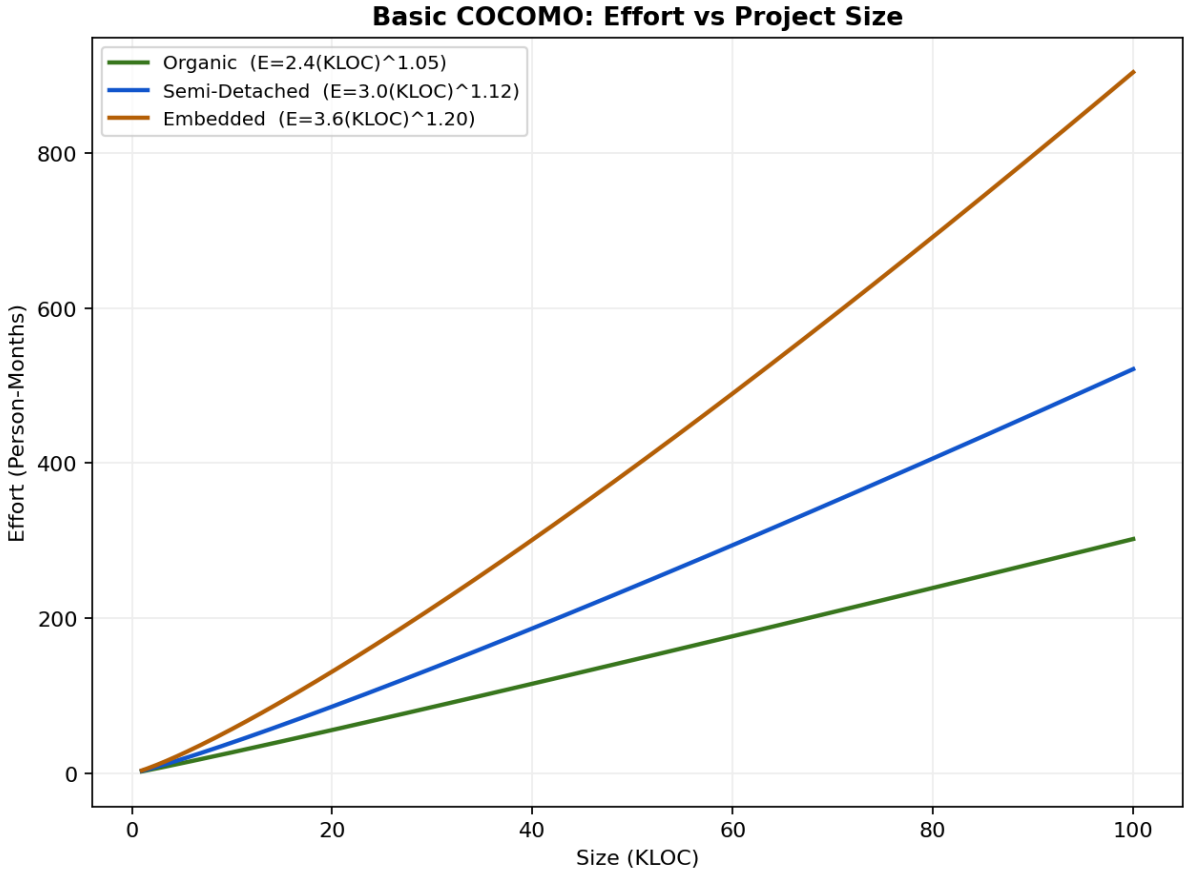

Figure: Basic COCOMO Effort vs Project Size

15. Attributes of Good Software / Components of Software Characteristics *[4 Marks]*

See the 2021 Examinations section, Q1, for the complete list and explanation. In brief, the key attributes of good software are: Functionality, Reliability, Usability, Efficiency, Maintainability, Portability, Security, and Correctness — together these determine whether software is fit for its intended purpose and easy to sustain over its lifetime.

Extra Questions

These questions were included in your provided list but did not appear directly in the collected past exam papers. Full answers are provided below for completeness.

Q1. Explain specification qualities. Explain operational specification with its types. *[8 Marks]*

In Simple English

A good specification needs certain qualities — like being clear and complete — to actually be useful. An operational specification is one particular style of writing a specification, describing the system in terms of how it behaves/operates.

Qualities of a Good Specification

- **Correctness:** Accurately reflects the actual needs of the stakeholders.
- **Completeness:** Includes all significant requirements, with no missing functionality or constraints.
- **Unambiguity:** Each requirement has only one possible interpretation.
- **Consistency:** No requirement conflicts with another requirement in the same document.
- **Verifiability:** Each requirement can be checked/tested to confirm it has been satisfied.
- **Traceability:** Each requirement can be traced back to its origin (e.g., a business need) and forward to its implementation/test case.
- **Modifiability:** The specification is structured so that changes can be made easily and consistently.

Operational Specification

An operational specification describes a system's required behavior directly in terms of a model that can be 'operated' or executed (simulated) to observe the behavior it produces, rather than describing behavior through purely descriptive, non-executable statements. It effectively specifies a system by showing how it operates, allowing early validation of requirements through simulation or prototyping.

Types of Operational Specifications

- **State-Transition Based:** Specifies behavior in terms of states and the transitions between them triggered by events (e.g., finite state machines).
- **Petri-Net Based:** Uses Petri nets to model concurrent and asynchronous behavior, useful for systems with parallel processes.
- **Algebraic/Formal Executable Specifications:** Use formal, mathematically-executable notations that can be directly simulated or 'run' to check expected behavior against requirements.
- **Prototype-Based:** A working prototype itself serves as an operational specification, letting stakeholders interact with and validate a simulated version of the intended system.

Q2. What are different test case design techniques? *[8 Marks]*

In Simple English

Test case design techniques are systematic methods for choosing which specific inputs to test with, so that testers can catch the most bugs without having to try literally every possible input.

Technical Explanation — Major Test Case Design Techniques

- **Equivalence Partitioning:** Divides input data into partitions (groups) that are expected to be treated the same way by the system, testing just one representative value from each partition rather than every possible value.
- **Boundary Value Analysis:** Focuses on testing values at the edges/boundaries of input ranges (minimum, maximum, just inside/outside limits), since errors often occur at boundary conditions.
- **Decision Table Testing:** Uses a table to systematically capture different combinations of conditions and their corresponding expected actions, useful for complex business logic.
- **State Transition Testing:** Designs test cases based on the different states a system can be in and the transitions between them, ensuring all states and transitions are exercised.
- **Use Case Testing:** Derives test cases directly from documented use cases, covering the main flow and alternate/exception flows.
- **Statement/Branch/Path Coverage (White-Box):** Designs test cases to ensure every statement, branch, or path in the code is executed at least once.
- **Error Guessing:** Relies on tester experience/intuition to guess likely problem areas or common mistakes not covered by other systematic techniques.

Q3. Define reliability and availability in context of a software reliability model diagram. *[8 Marks]*

In Simple English

Reliability is about how likely the software is to keep working without failing; availability is about how much of the time it's actually usable, including any time lost to repairs.

Definitions

Reliability: The probability that software will perform its required functions, without failure, for a specified period of time under specified operating conditions.

Availability: The probability that software is operational and available for use at any given point in time, accounting for both failures and the time required to repair them.

Software Reliability Model Diagram

A software reliability model typically plots the cumulative number of failures (or failure intensity) against time or execution period, showing how the failure rate decreases as defects are discovered and fixed during testing (a 'reliability growth' curve). Key relationships shown/derived from such a model include:

```
MTBF = Total Operating Time / Number of Failures
Reliability R(t) = e^(-λt)     where λ = failure rate, t = time period
Availability = MTBF / (MTBF + MTTR)
```

As testing/operation continues and more defects are fixed, the failure rate (λ) decreases, causing the reliability curve to flatten out at a higher reliability value over time — this growth pattern is the basis of well-known reliability growth models such as the Jelinski-Moranda model and the Musa-Okumoto model, which are used to predict when a software product is reliable enough for release.